

GCOS108

Clustering

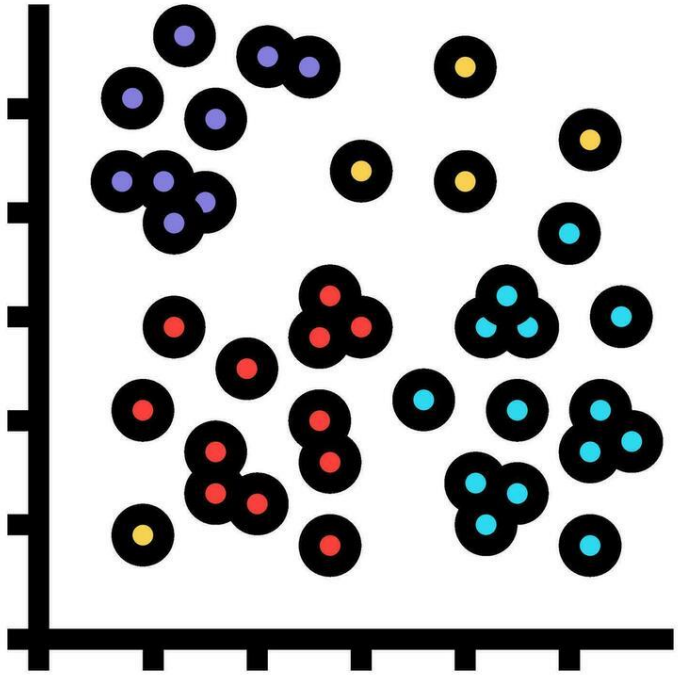
Trinh D.D. **NGUYEN** (*Ph.D.*), Bay **VO** (*Assoc. Prof.*)

FIT - HUTECH

REV.10-26

Outline

- Motivations
- Modelling and distance measures
- *k*-Means
- *k*-Means++
- *k*-Medoids
- Wrapping up



Learning objectives

- Explain what **clustering** is and where it is used.
- Model clustering as an **optimisation problem**.
- Choose a suitable **distance measure**.
- Run **k-Means**, **k-Means++** and **k-Medoids** by hand.
- Judge a clustering and choose **k**.



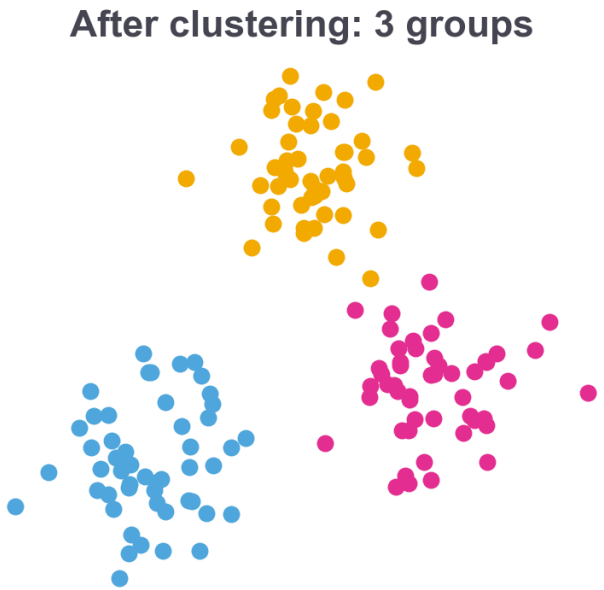


Motivations

Finding structure in data that nobody labelled

Data without labels

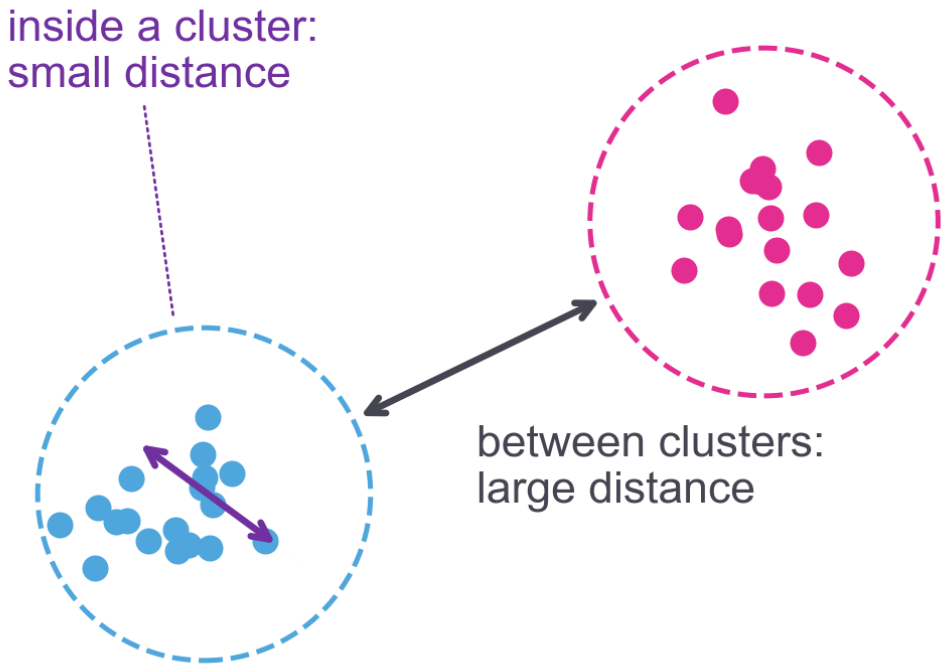




- Most data come **without labels**.
- Is there still **structure** in it?
- Idea: put **similar** objects in the same group.

What is clustering?

- Group objects into **clusters** so that
  - objects in the same cluster are **similar**;
  - objects in different clusters are **dissimilar**.
- It is **unsupervised learning**: no predefined classes.



Classification vs. Clustering

Classification

- **Supervised**: class labels are given.
- Learn a mapping from attributes to a label.
- Evaluate with accuracy, F1…
- Decision trees, ILA.

Clustering

- **Unsupervised**: no labels at all.
- Discover the groups themselves.
- Evaluation is indirect.
- k-Means, k-Medoids, DBSCAN…

Applications of clustering

Marketing
Customer segments for targeted campaigns

Documents, Web
Group news, search results, access logs

Biology
Genes and proteins with similar function

Images
Segmentation, colour quantisation

Finance, insurance
Stocks that move together, risky policy groups

Cities, Earth
Land use, house groups, earthquakes along faults

Two ways clustering is used

Understanding

- Stand-alone tool.
- What groups exist?
- What characterises each?

Summarisation

- Replace many objects by a few representatives.
- Compression, sampling.

Preprocessing

- Before classification or search.
- Outlier detection: what fits no cluster?

Real world: customer segmentation



- Each customer: **recency**, **frequency**, **spend**.
- Segments appear that **nobody defined in advance**.
- One campaign per segment.

Real world: colour quantisation

Original (25,071 colours)



k-Means, k = 16 colours



k-Means, k = 4 colours



Every pixel is a point in RGB space.
k-Means keeps only **k representative colours**:
25,071 colours become 16, then 4.

Real world: clustering at scale

Sky survey

- 2 billion sky objects
- 9 dimensions (frequency bands)
- Stars, galaxies, quasars

Documents

- A document = a vector of words
- Similar words → same topic
- Millions of articles

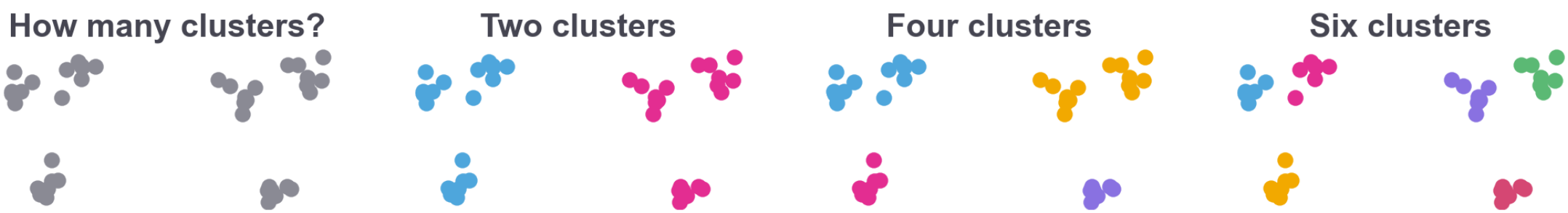
Recommendation

- A product = the customers who bought it
- Similar buyers → similar products

What makes a clustering method good?

- High similarity inside clusters
- Low similarity between clusters
- Scales to large data
- Handles many attribute types
- Finds clusters of any shape
- Robust to noise and outliers
- Insensitive to input order
- Works in high dimensions
- Results are interpretable

The notion of a cluster is ambiguous



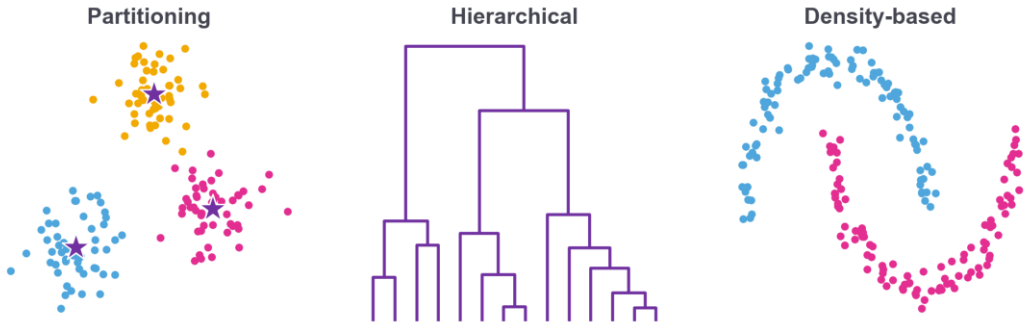
Two, four or six clusters?

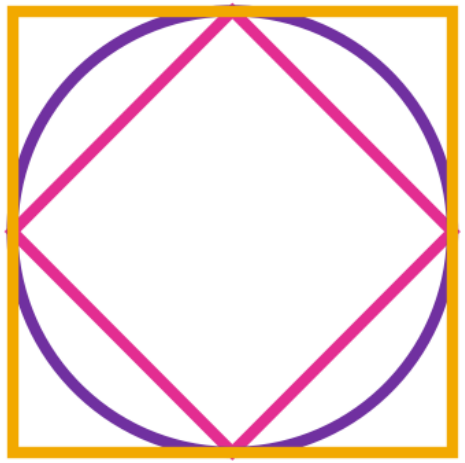
The answer depends on the goal, the distance and the method.

Major clustering approaches

Approach	Idea	Examples
Partitioning	*k* groups that optimise a criterion	*k*-Means, *k*-Medoids
Hierarchical	a tree of nested clusters	AGNES, DIANA, BIRCH
Density-based	dense regions separated by sparse ones	DBSCAN, OPTICS
Grid-based	cluster the cells of a grid	STING, CLIQUE
Model-based	fit a model to each cluster	EM, SOM

Today: **partitioning methods**.





Modelling and distance measures

From objects to numbers, from similarity to an objective

Two ways to store the data

Data matrix (*n* objects × *p* attributes)

	A_1	A_2	…	A_p
x_1	x_{11}	x_{12}	…	x_{1p}
x_2	x_{21}	x_{22}	…	x_{2p}
…	…	…	…	…
x_n	x_{n1}	x_{n2}	…	x_{np}

Dissimilarity matrix (*n* × *n*)

	x_1	x_2	x_3	…
x_1	0			
x_2	d(2,1)	0		
x_3	d(3,1)	d(3,2)	0	
…	…	…	…	0

k-Means works on the data matrix.
k-Medoids only needs the **dissimilarities** $d(i,j)$.

Clustering as an optimisation problem

A partition of *D* into *k* clusters

$$\bigcup_{i=1}^{k} C_i = D, \qquad C_i \cap C_j = \emptyset \ \ (i \neq j), \qquad C_i \neq \emptyset$$

Goal

$$\mathcal{C}^* = \operatorname{argmin}_{\mathcal{C}} J(\mathcal{C})$$

- J measures how **bad** a partition is, for example the total distance inside the clusters.
- Each choice of J gives an algorithm.

How many partitions are there?

Ways to split *n* objects into *k* clusters

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^j\binom{k}{j}(k-j)^n \approx \frac{k^n}{k!}$$

- **Exhaustive search is impossible.**
- We need **heuristics**: start from a partition and improve it.

n	k	partitions
8	3	966
10	3	9330
20	4	4.5×10^{10}
25	4	4.7×10^{13}
100	5	6.6×10^{67}

What is a distance?

$d(x, y) \geq 0$	never negative
$d(x, y) = 0 \Leftrightarrow x = y$	zero only for identical objects
$d(x, y) = d(y, x)$	symmetric
$d(x, z) \leq d(x, y) + d(y, z)$	triangle inequality

A function with these four properties is a **metric**.

The Minkowski family

Minkowski distance of order *q*

$$d(i,j) = \left(\sum_{f=1}^{p} |x_{if} - x_{jf}|^q \right)^{1/q}$$

$q = 1$: Manhattan

$$d = \sum_{f} |x_{if} - x_{jf}|$$

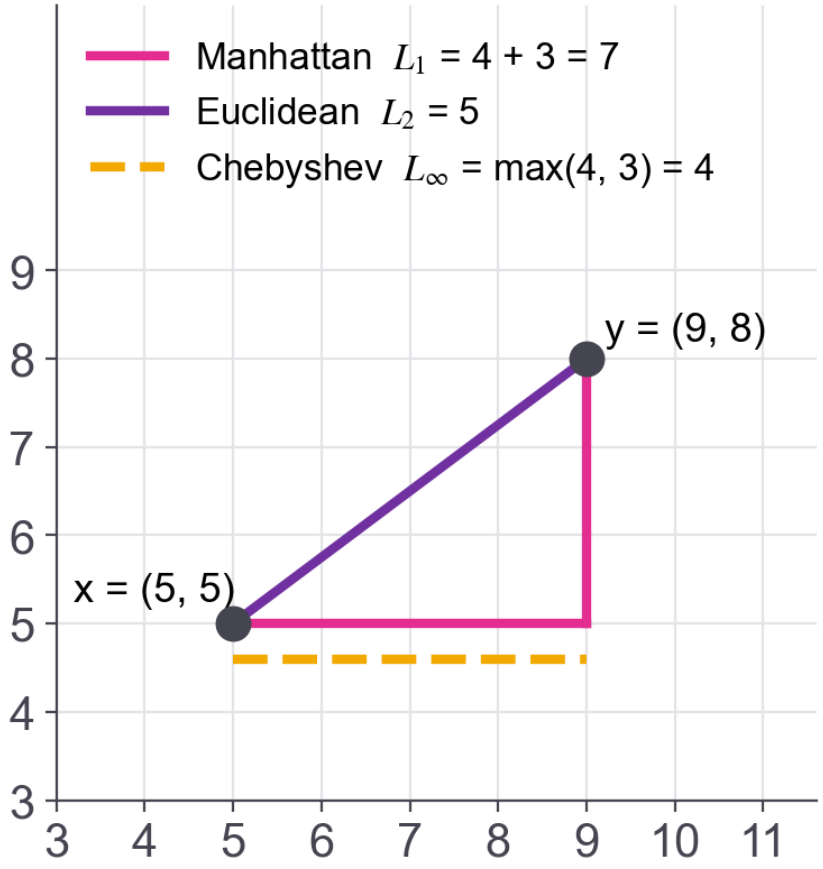
$q = 2$: Euclidean

$$d = \sqrt{\sum_{f} (x_{if} - x_{jf})^2}$$

$q \to \infty$: Chebyshev

$$d = \max_{f} |x_{if} - x_{jf}|$$

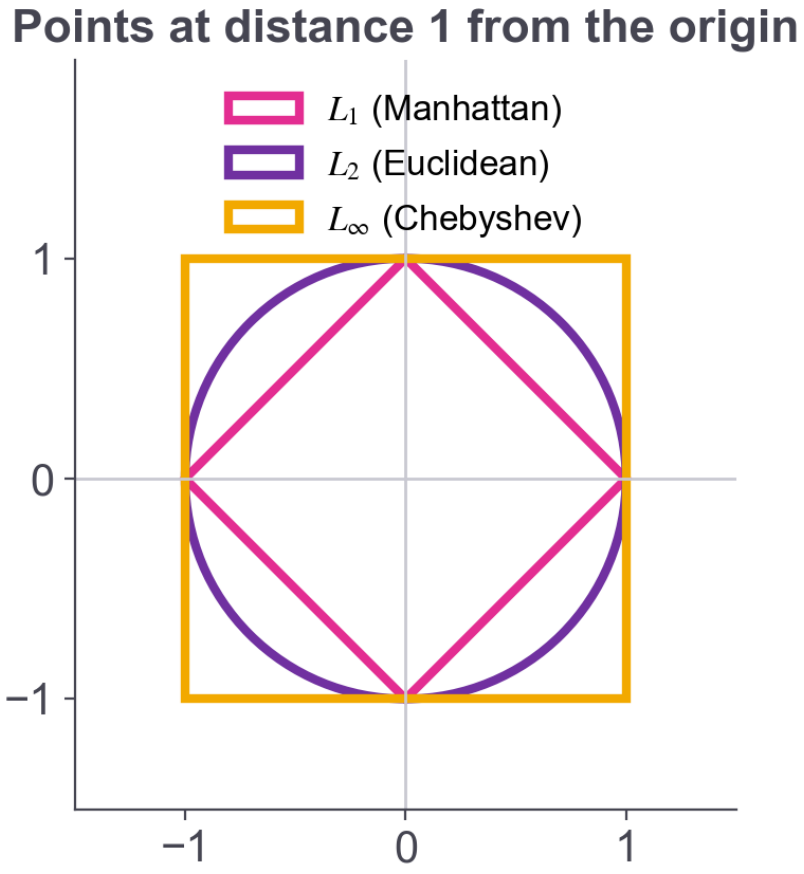
Three distances, one pair of points



Distance	Computation	Value
Manhattan	$\|5-9\| + \|5-8\|$	**7**
Euclidean	$\sqrt{(4^2 + 3^2)}$	**5**
Chebyshev	max(4, 3)	**4**

Always: $L_\infty \leq L_2 \leq L_1$.

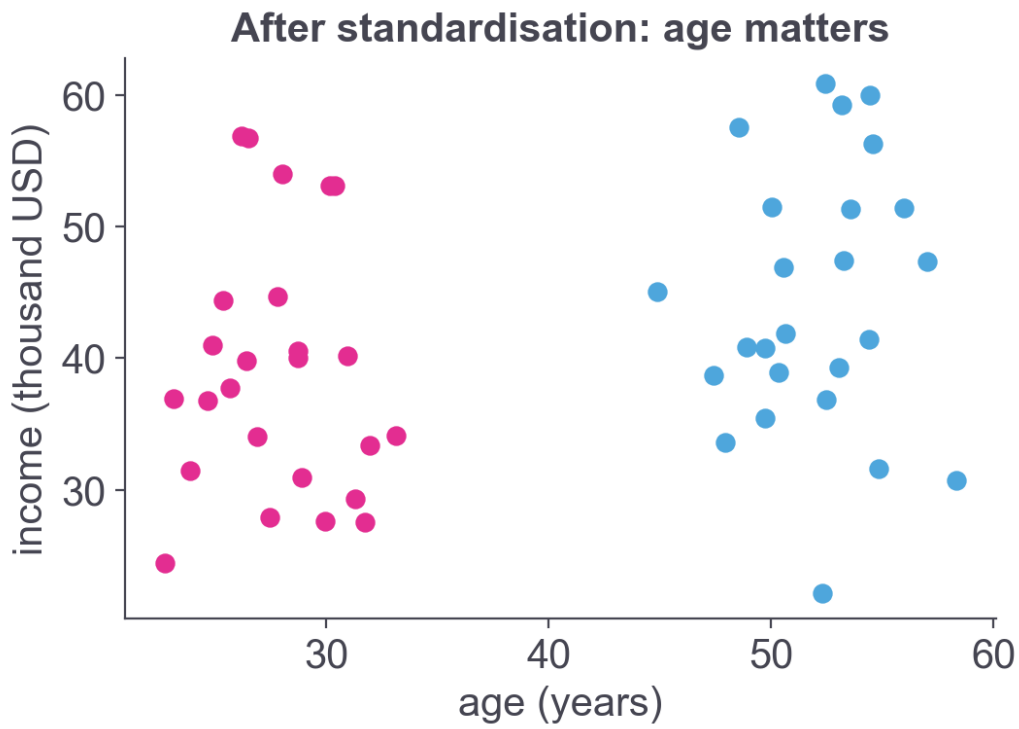
The shape of “distance 1”



- **Manhattan**: travel along the axes.
- **Euclidean**: the straight line.
- **Chebyshev**: the largest coordinate gap.
- The distance decides who is a **neighbour**.

Units matter: standardise first





Income in dollars swamps age in years: **the larger unit wins**.

Standardising an attribute

Mean absolute deviation and *z*-score (m_f = mean of attribute *f*)

$$s_f = \frac{1}{n}\sum_{i=1}^{n} |x_{if} - m_f| \qquad z_{if} = \frac{x_{if} - m_f}{s_f}$$

age	25	30	45	50
***z*-score**	−1.25	−0.75	0.75	1.25

$$m = 37.5, \qquad s = 10$$

Why absolute deviation?

Deviations are not squared, so the measure is **more robust** to outliers than the standard deviation.

Binary attributes

	$j = 1$	$j = 0$
$i = 1$	q	r
$i = 0$	s	t

Count the attributes in each cell for objects i and j.

Symmetric: simple matching

$$d(i,j) = \frac{r+s}{q+r+s+t}$$

Asymmetric: Jaccard (ignore 0–0 matches)

$$d(i,j) = \frac{r+s}{q+r+s}$$

Asymmetric: a 1 (a positive test) carries information, a 0 does not.

Example: three patients

	Fever	Cough	Test 1	Test 2	Test 3	Test 4
Jack	Y	N	P	N	N	N
Mary	Y	N	P	N	P	N
Jim	Y	P	N	N	N	N

Pair	Jaccard *d*
Jack, Mary	**0.33**
Jack, Jim	0.67
Jim, Mary	0.75

$$d(\text{Jack}, \text{Mary}) = \frac{0 + 1}{2 + 0 + 1} = 0.33$$

Jack and Mary are the **most alike**: probably the same disease.

Hamming distance

Number of positions in which *x* and *y* differ

$$d_H(x, y) = |\{f : x_f \neq y_f\}|$$

x	y	Hamming
1 0 1 1 1	1 0 0 1 1	**1**
k a r o l i n	k a t h r i n	**3**
Jack	Jim	**2**

- For 0/1 vectors: Hamming = Manhattan = $r + s$.
- The distance for **categorical** data.

Nominal and ordinal attributes

Nominal (colour, city...)

$$d(i,j) = \frac{p - m}{p}$$

m matching attributes out of p.

red vs. blue: mismatch; red vs. red: match.

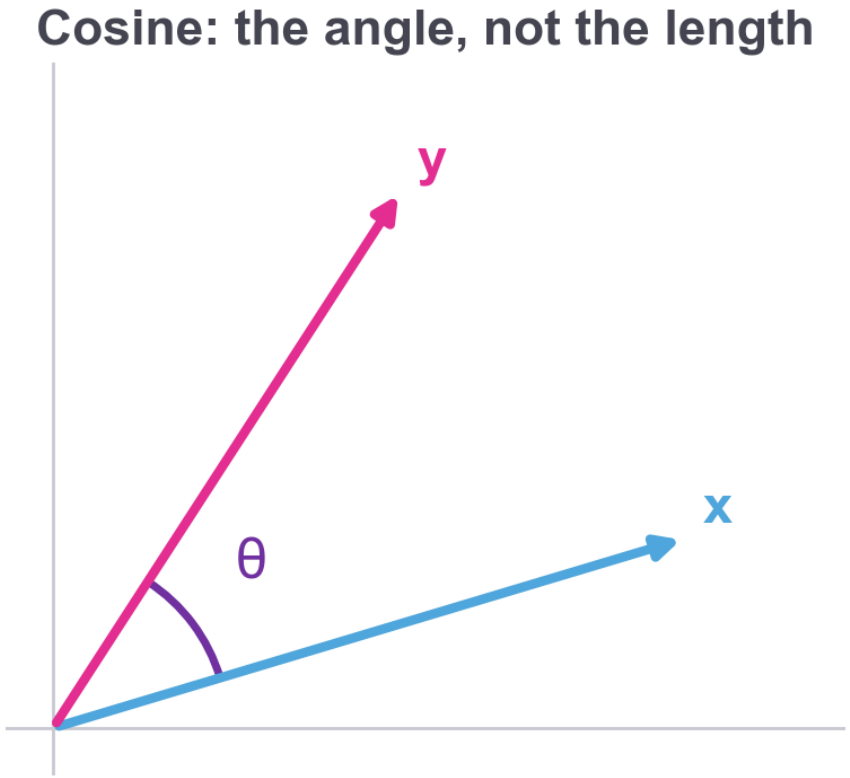
Ordinal (low < med < high)

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

Replace the value by its rank r_{if} among M_f levels, then use a numeric distance.

low, medium, high → 0, 0.5, 1

Cosine similarity for documents



$$\cos(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$

x = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0)
y = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1)

$$\cos(x, y) = \frac{25}{6.48 \times 4.12} = 0.94$$

The **length** of a document does not matter, only its direction.

Distances beyond vectors

Jaccard distance (sets)

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

A = {1, 3, 4, 5}, B = {1, 4, 5}

d = 1 − 3/4 = **0.25**

Edit distance (strings)

Fewest inserts and deletes turning one string into the other.

abcde → bcduve

delete a, insert u, insert v:

d = **3**

No “average object” exists here: **no mean, no k-Means**. *k*-Medoids still works.

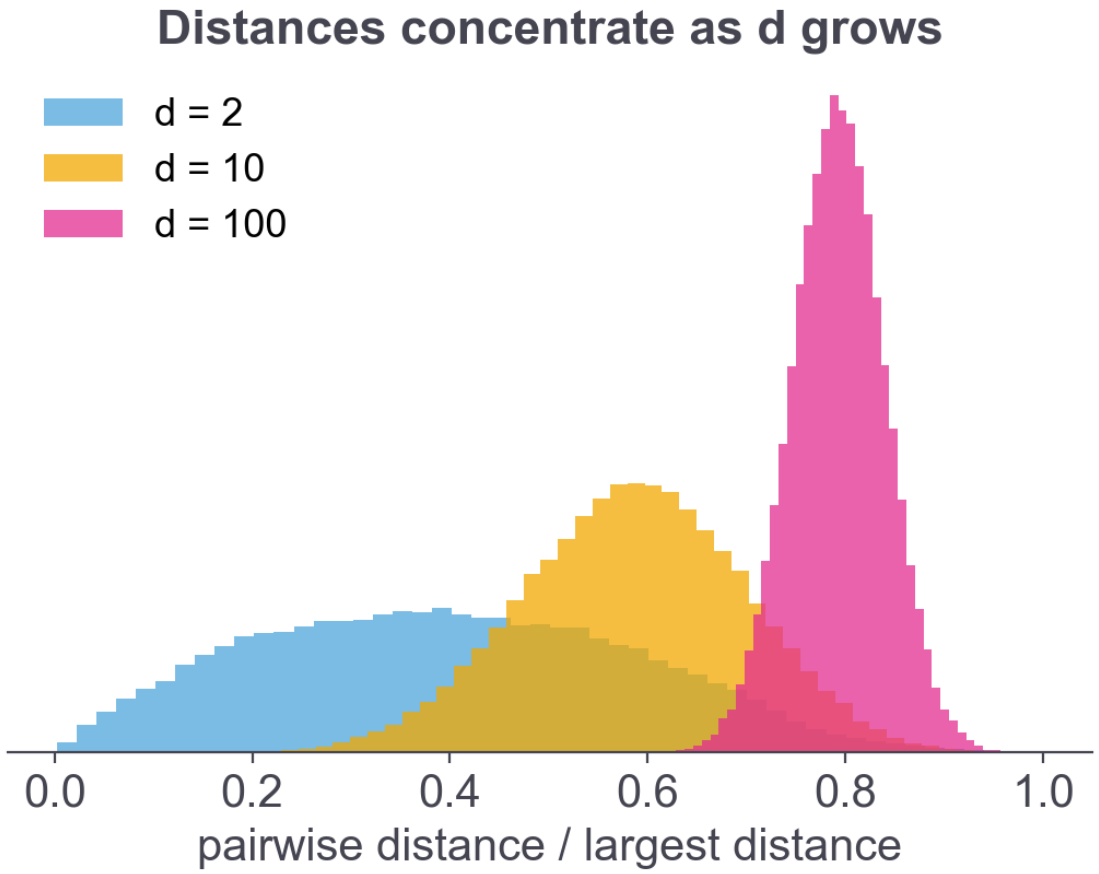
Attributes of mixed types

Average the per-attribute dissimilarities

$$d(i,j) = \frac{\sum_{f=1}^{p} \delta_{ij}^{(f)} \, d_{ij}^{(f)}}{\sum_{f=1}^{p} \delta_{ij}^{(f)}}$$

- Numeric: $|x_{if} - x_{jf}|$ divided by the range.
- Binary or nominal: 0 if equal, 1 otherwise.
- Ordinal: through the ranks.
- $\delta = 0$ when a value is **missing**.

The curse of dimensionality



- In high dimensions **all points look equally far**.
- “Nearest” and “farthest” blur.
- Reduce or select dimensions first.

Which distance when?

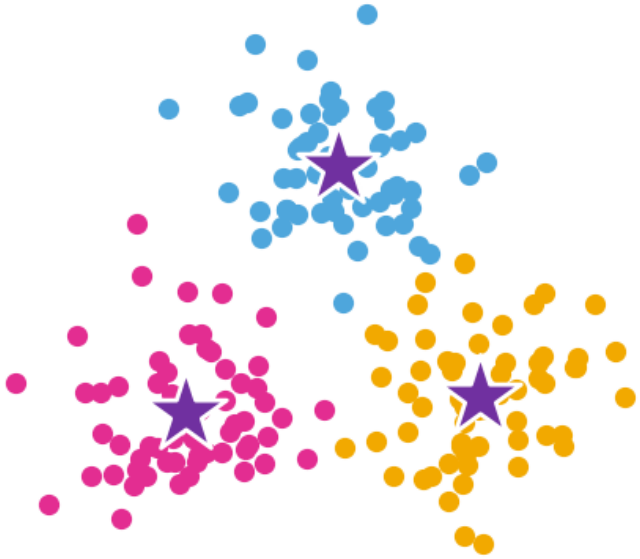
Data	Typical measure
numeric, comparable scales	Euclidean
numeric, outliers or grid-like moves	Manhattan
binary, both values informative	simple matching, Hamming
binary asymmetric, sets	Jaccard
documents, sparse high-dimensional	cosine
strings, sequences	edit distance
mixed types	Gower combination

Exercise 1: distances

1. x = (1, 2), y = (4, 6): compute the **Manhattan**, **Euclidean** and **Chebyshev** distances.

2. a = 110010, b = 100111: compute the **Hamming** distance and the **Jaccard** dissimilarity.

3. Standardise the values 15, 25, 30, 50 with the mean absolute deviation.

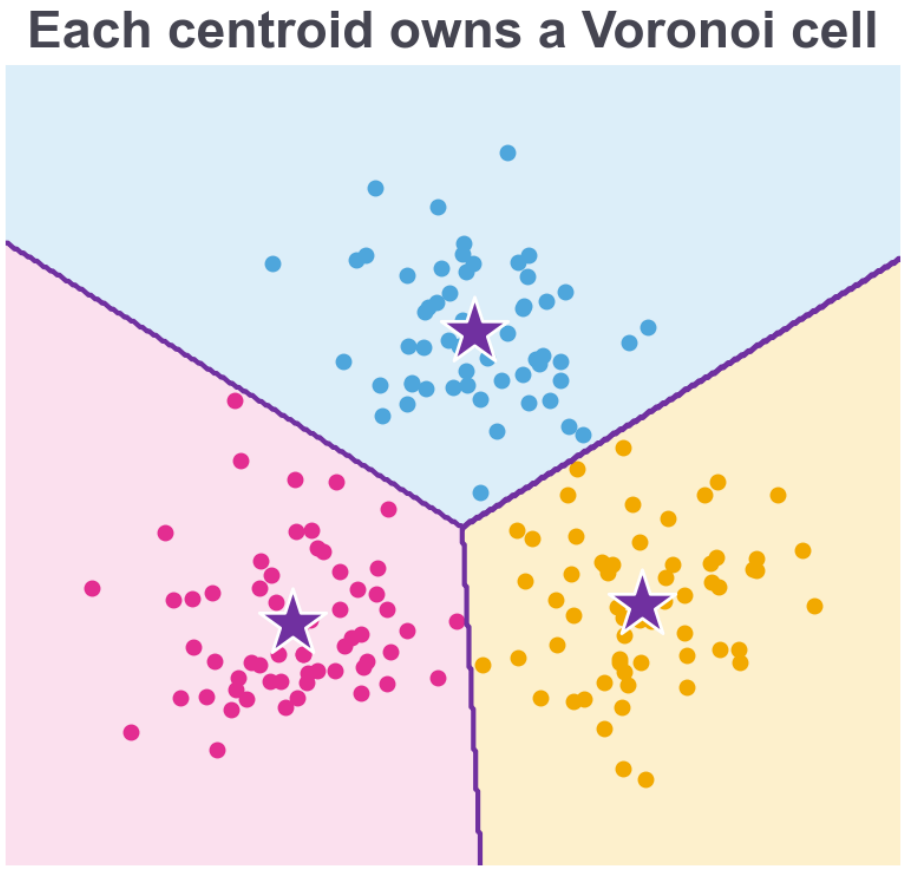


k-Means

The most widely used clustering algorithm

Partitioning with representatives

- Fix the number of clusters **k** in advance.
- Represent each cluster by one point: its **centroid**.
- Every object joins the **nearest** centroid.



The *k*-Means objective

Centroid of cluster i

$$\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\mathbf{x}_j \in C_i} \mathbf{x}_j$$

Sum of squared errors

$$SSE(\mathcal{C}) = \sum_{i=1}^{k} \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2$$

- SSE adds the squared distance of every object to **its own centroid**.
- Goal: the partition with the **smallest SSE**.

Why the mean?

Best representative of one cluster

$$f(\mathbf{c}) = \sum_{\mathbf{x}_j \in C} \|\mathbf{x}_j - \mathbf{c}\|^2 \qquad \frac{\partial f}{\partial \mathbf{c}} = -2 \sum_{\mathbf{x}_j \in C} (\mathbf{x}_j - \mathbf{c}) = 0$$

Solution: the mean

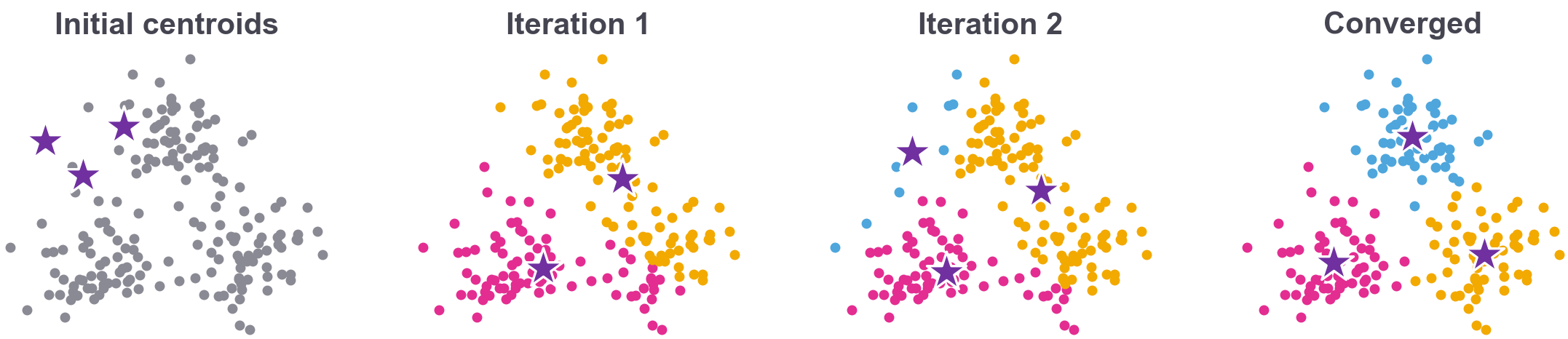
$$\mathbf{c} = \frac{1}{|C|} \sum_{\mathbf{x}_j \in C} \mathbf{x}_j$$

- The mean is the **best** centre for squared Euclidean distance.
- Manhattan distance → the **median**.

The *k*-Means algorithm

1. Choose **k initial centroids** (for example, k random objects).
2. **Assign** every object to its nearest centroid.
3. **Update** every centroid to the mean of its cluster.
4. Repeat 2 and 3 until **no object changes cluster**.

k-Means at work



Assign, **update**, repeat: the centroids move until nothing changes.

Example 1: nine numbers on a line

Data: 2, 3, 4, 10, 11, 12, 20, 25, 30 k = 2 initial means $\mu_1 = 2, \mu_2 = 4$

Assign

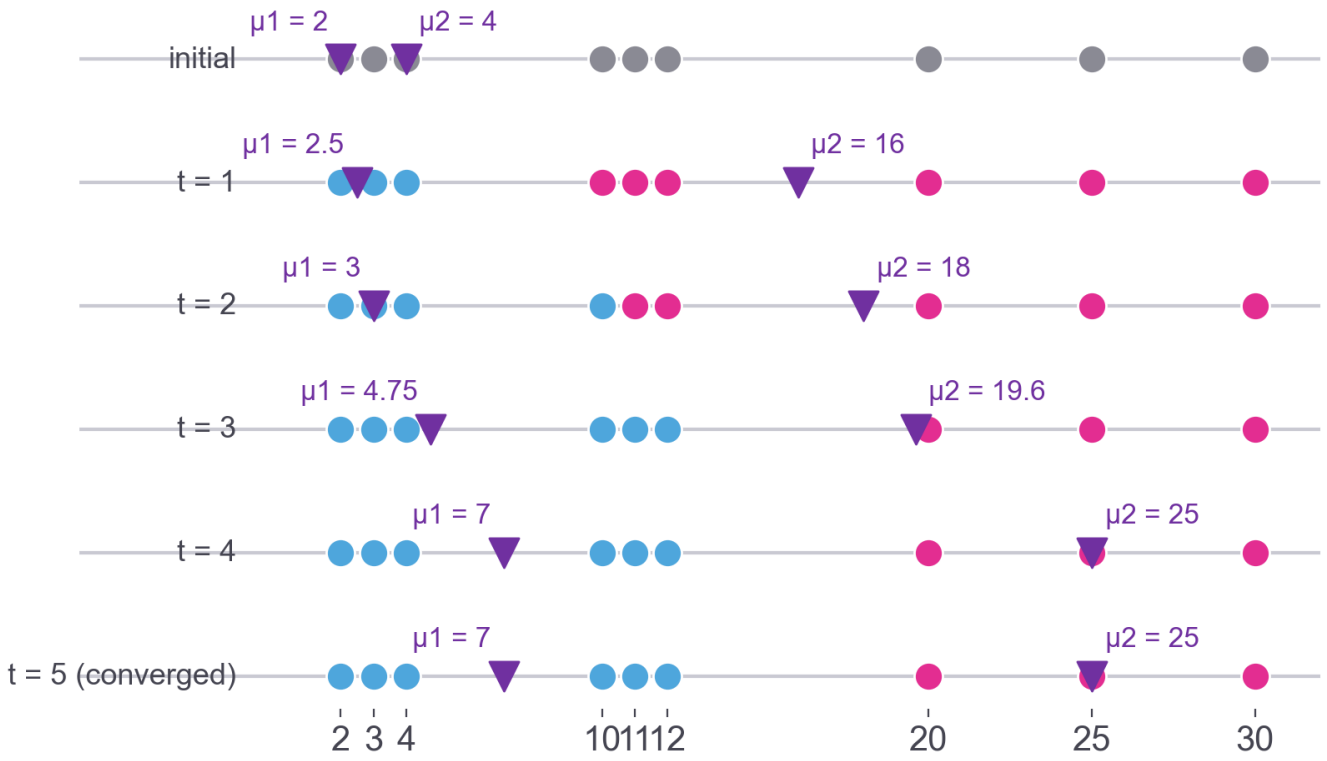
The midpoint of 2 and 4 is 3.

$$C_1 = \{2,3\}$$
$$C_2 = \{4,10,11,12,20,25,30\}$$

Update

$$\mu_1 = \frac{2+3}{2} = 2.5$$
$$\mu_2 = \frac{4+10+11+12+20+25+30}{7} = 16$$

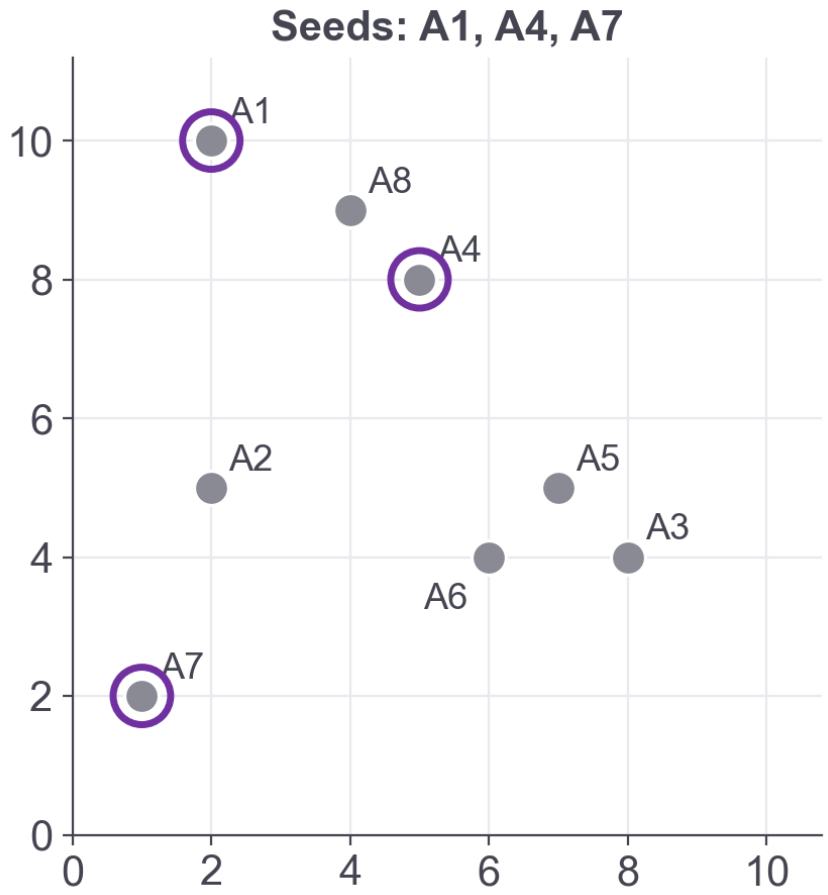
Example 1: the whole run



t	μ_1	μ_2	SSE
1	2.5	16	514.5
2	3	18	348
3	4.75	19.6	307.95
4	7	25	**150**
5	7	25	**150**

No change at t = 5: **stop**.

Example 2: eight points in the plane

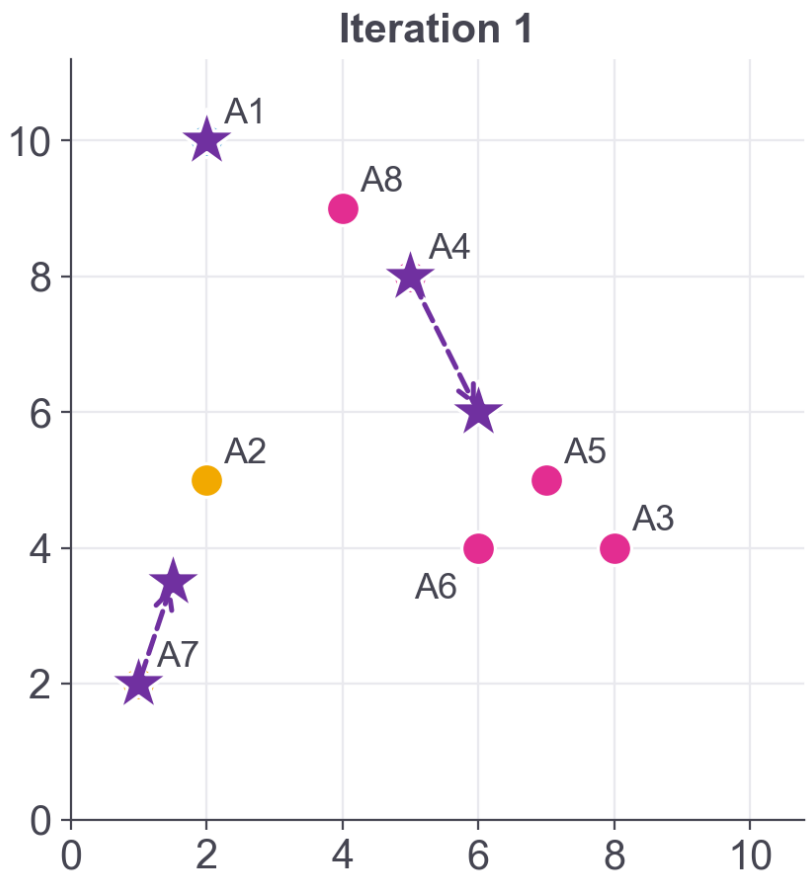


	x	y		x	y
A1	2	10	A5	7	5
A2	2	5	A6	6	4
A3	8	4	A7	1	2
A4	5	8	A8	4	9

- k = 3, **Euclidean** distance.
- Seeds: **A1**, **A4**, **A7**.

Example 2, iteration 1: assign

Point	d^2 to A1	d^2 to A4	d^2 to A7	Cluster
A1	0	13	65	C1
A2	25	18	10	C3
A3	72	25	53	C2
A4	13	0	52	C2
A5	50	13	45	C2
A6	52	17	29	C2
A7	65	52	0	C3
A8	5	2	58	C2



Example 2, iteration 1: update

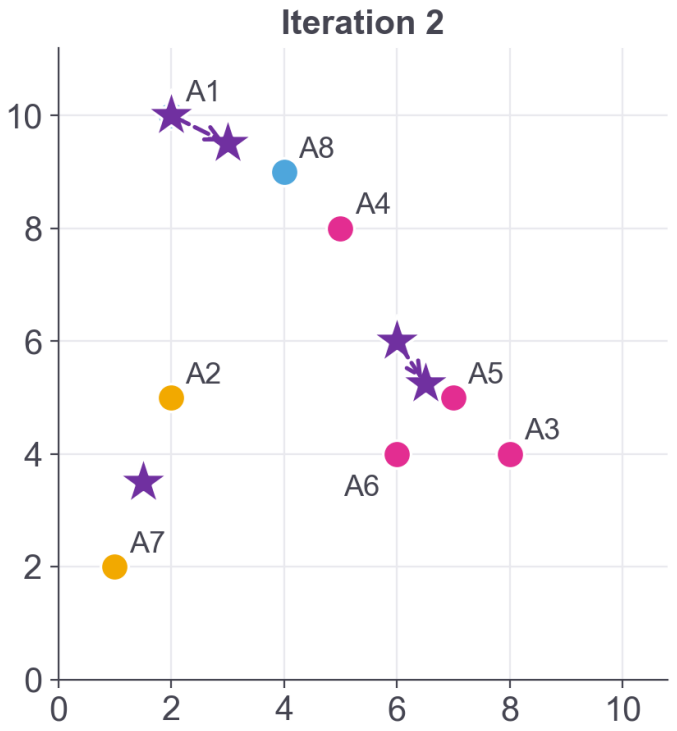
Cluster	Members	New centroid	SSE
C1	A1	(2, 10)	0
C2	A3, A4, A5, A6, A8	(6, 6)	32
C3	A2, A7	(1.5, 3.5)	5

$$c_2 = \left(\frac{8+5+7+6+4}{5}, \frac{4+8+5+4+9}{5}\right) = (6, 6)$$

$$SSE = 0 + 32 + 5 = 37$$

Example 2, iteration 2

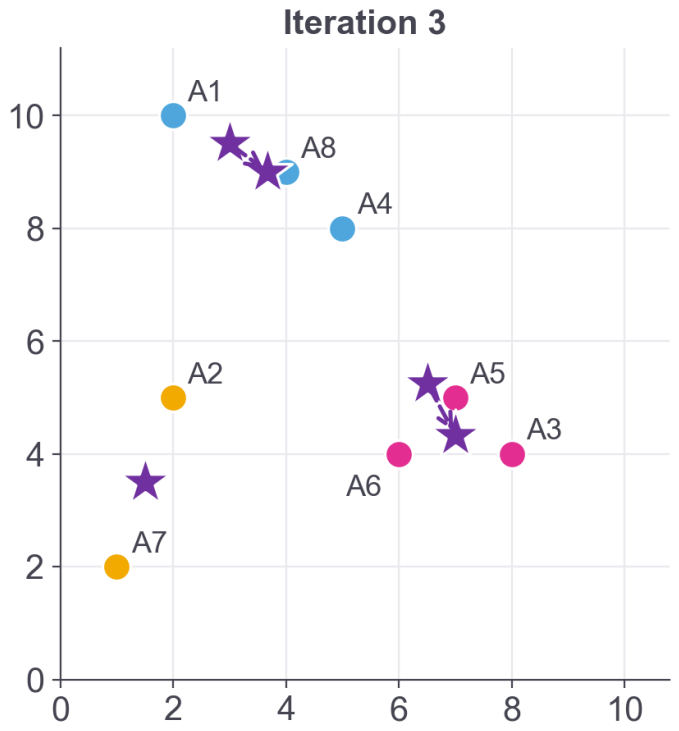
Point	d^2 to c1	d^2 to c2	d^2 to c3	Cluster
A1	0	32	42.5	C1
A2	25	17	2.5	C3
A3	72	8	42.5	C2
A4	13	5	32.5	C2
A5	50	2	32.5	C2
A6	52	4	20.5	C2
A7	65	41	2.5	C3
A8	5	13	36.5	C1



A8 moves to C1. SSE = **23.25**

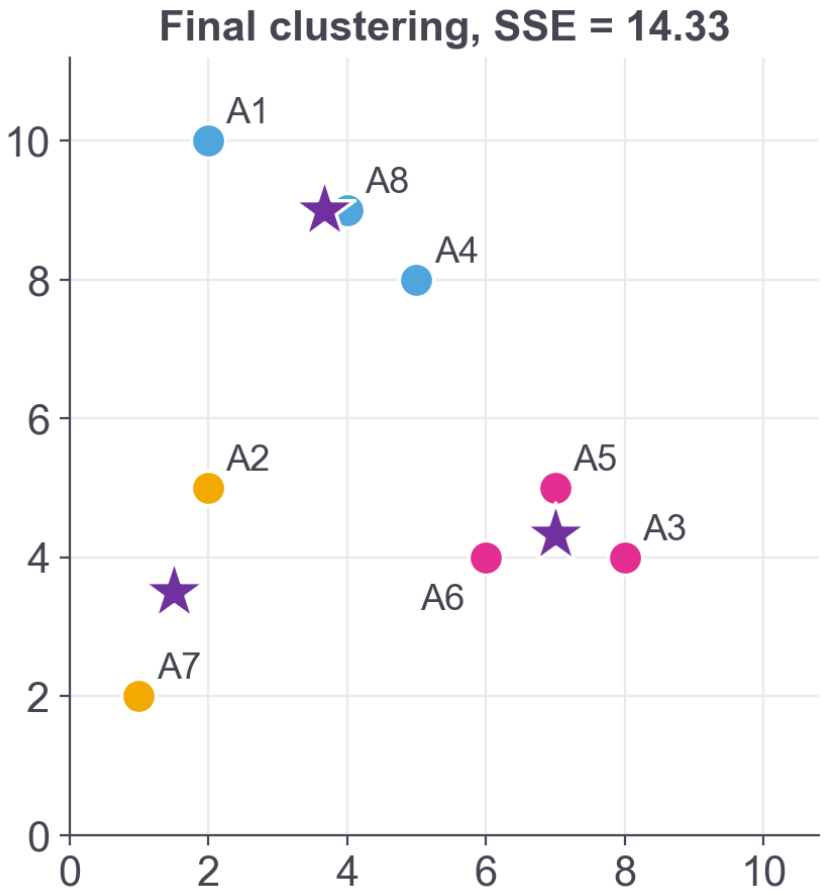
Example 2, iteration 3

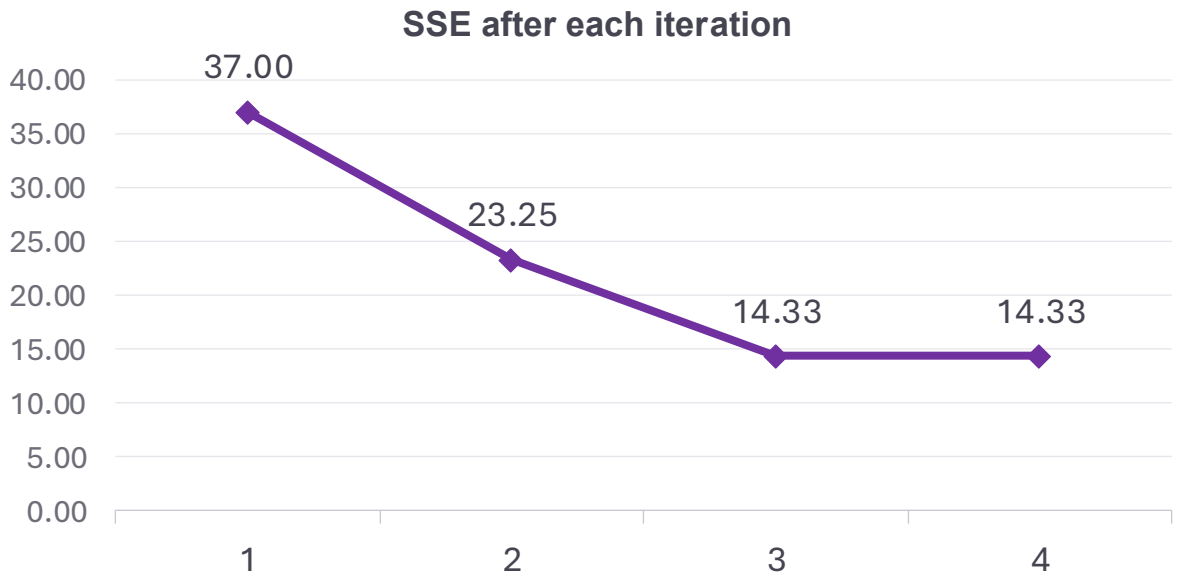
Point	d^2 to c1	d^2 to c2	d^2 to c3	Cluster
A1	1.25	42.81	42.5	C1
A2	21.25	20.31	2.5	C3
A3	55.25	3.81	42.5	C2
A4	6.25	9.81	32.5	C1
A5	36.25	0.31	32.5	C2
A6	39.25	1.81	20.5	C2
A7	60.25	40.81	2.5	C3
A8	1.25	20.31	36.5	C1



A4 moves to C1. SSE = **14.33**

Example 2: convergence





- Iteration 4: **no change** → stop.
- C1 = {A1, A4, A8}, C2 = {A3, A5, A6}, C3 = {A2, A7}

Why does *k*-Means stop?

1. **Assign**: each object goes to a nearer centroid, so SSE cannot increase.
2. **Update**: the mean is the best centre, so SSE cannot increase.
3. Only finitely many partitions exist, so the algorithm **must stop**.
4. **It stops at a local optimum, not necessarily the best one.**

Cost of *k*-Means

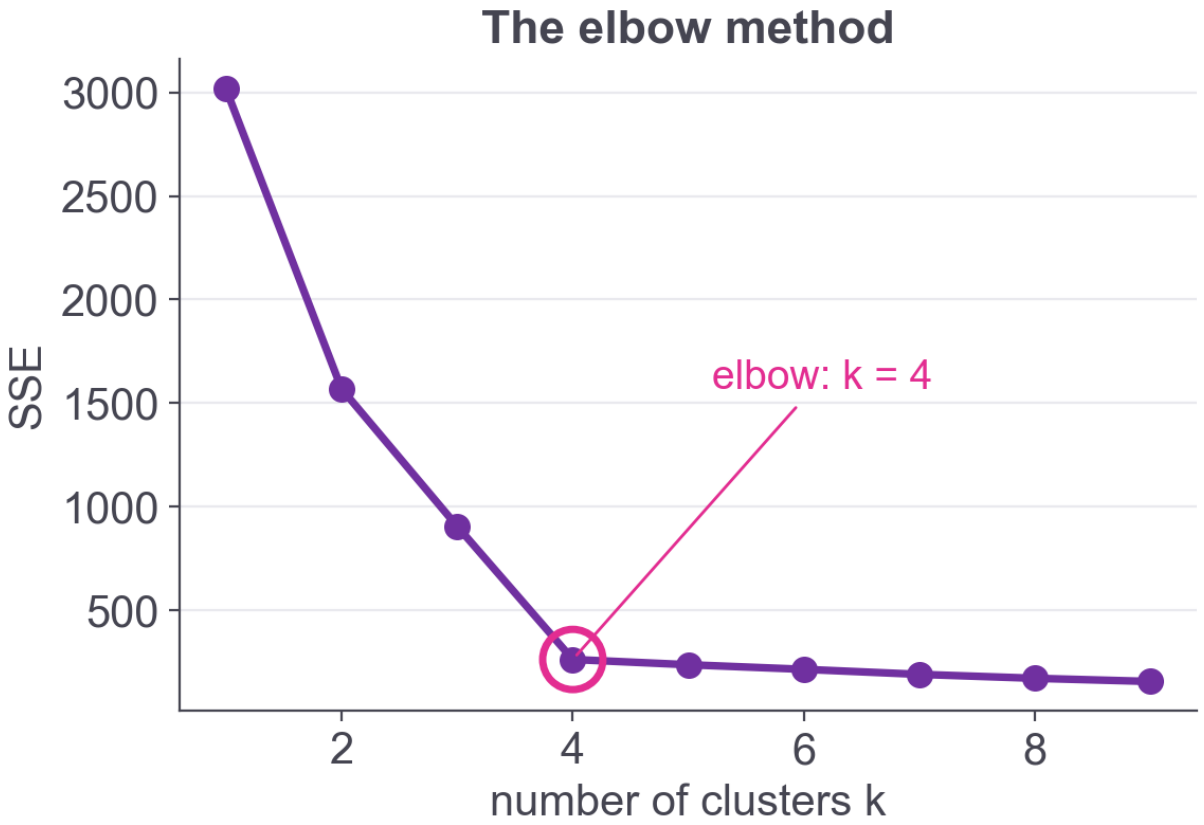
Time complexity

$$O(t \cdot k \cdot n \cdot d)$$

n: objects | **k: clusters** | **d: dimensions** | **t: iterations**

Linear in n: *k*-Means scales to millions of objects. Usually k, t $\ll$ n.

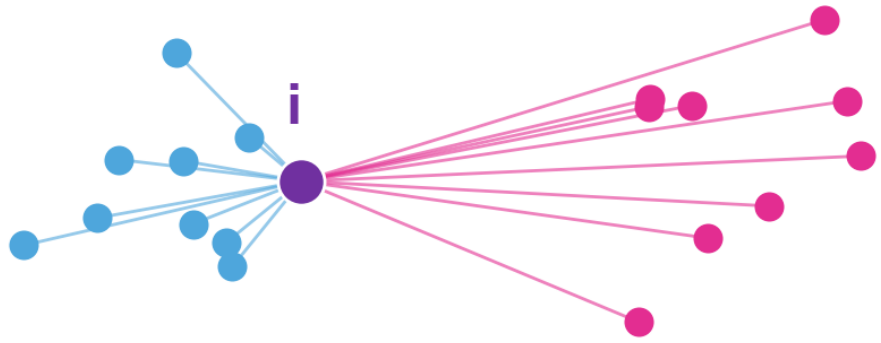
How many clusters? The elbow method



- Run *k*-Means for k = 1, 2, 3, …
- SSE **always** falls as k grows.
- Choose the **elbow**: beyond it, more clusters buy little.

k	1	2	3	4
SSE (Ex. 2)	100.75	51.47	**14.33**	8.67

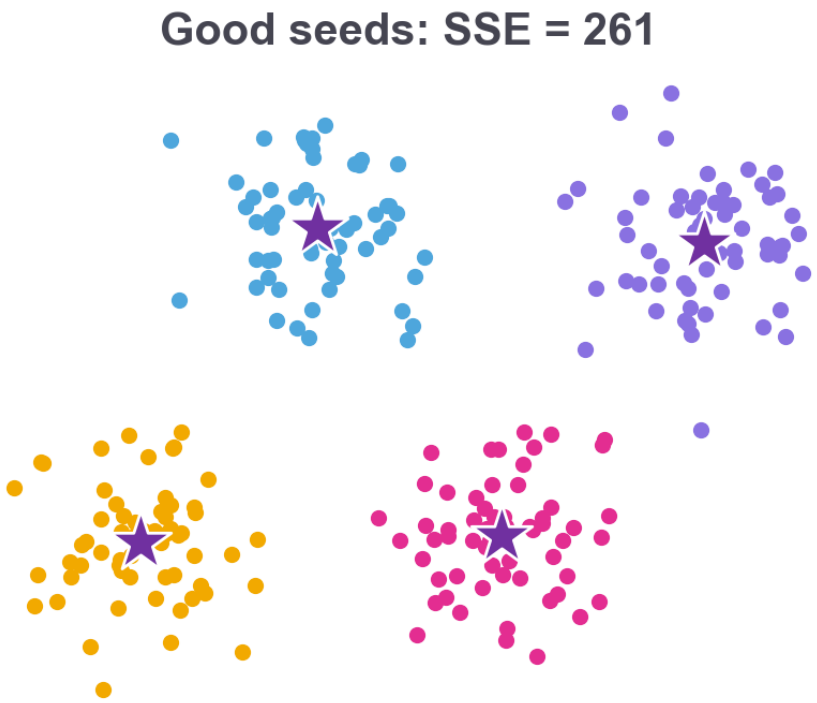
The silhouette coefficient

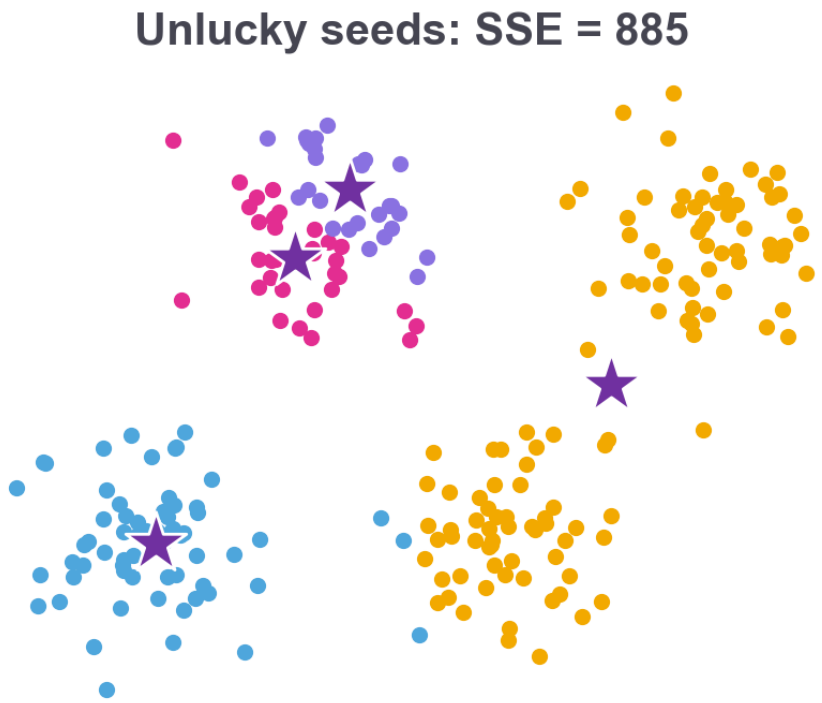


$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

- Near **1**: well placed.
- Near **0**: on a border.
- **Negative**: wrong cluster?
- Example 2: mean s = **0.57**

Initial centroids matter

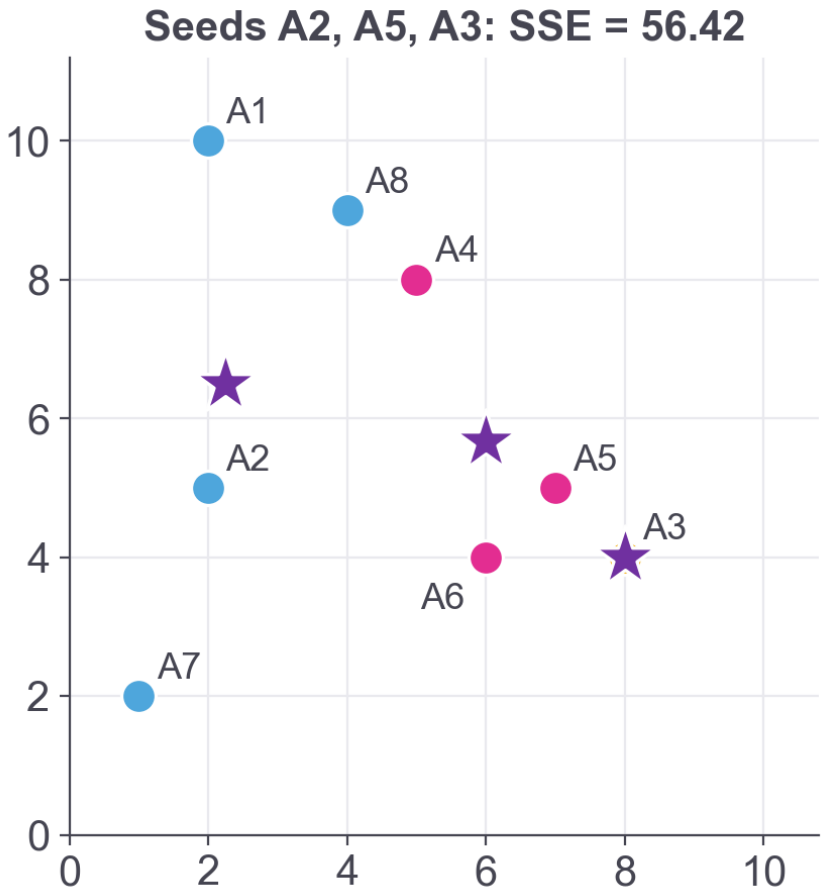




Same data, same k: SSE 261 or 885.

***k*-Means only finds a local optimum.**

Example 2 again, with unlucky seeds



Seeds	Iterations	Final SSE
A1, A4, A7	4	**14.33**
A2, A5, A3	2	56.42

- It stops **immediately**, four times worse.
- A3 keeps a cluster for itself; A1 and A7 are forced together.

How likely is a good start?

k equal-sized clusters, k random seeds

$$P(\text{one seed in every cluster}) = \frac{k!}{k^k}$$

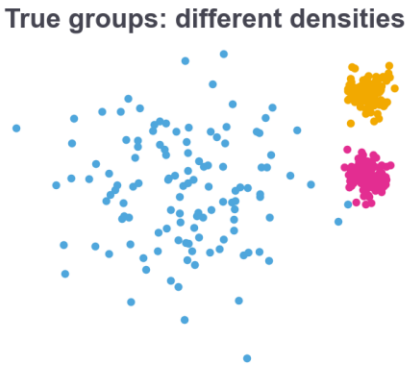
- With 10 clusters a random start is **almost never** good.
- Restarts help, but we need **better seeds**.

k	probability
2	0.50
3	0.22
5	0.038
10	**0.00036**

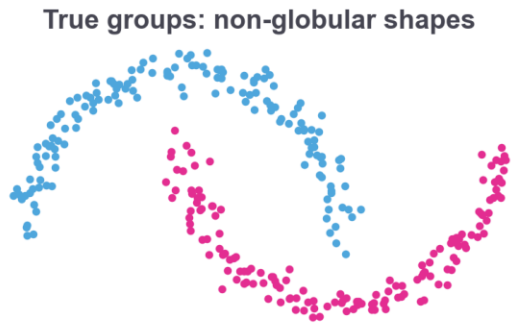
Limitations of *k*-Means

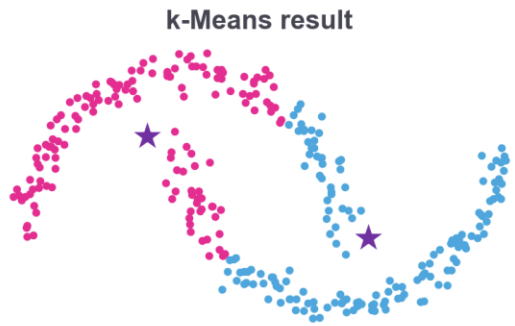












- Different **sizes**
- Different **densities**
- **Non-globular** shapes
- **Outliers**
- *k* must be given

Getting more out of *k*-Means

Restart

Run several times, keep the **lowest SSE**.

Seed carefully

k-Means++: the next part.

Prepare the data

Standardise; remove outliers first.

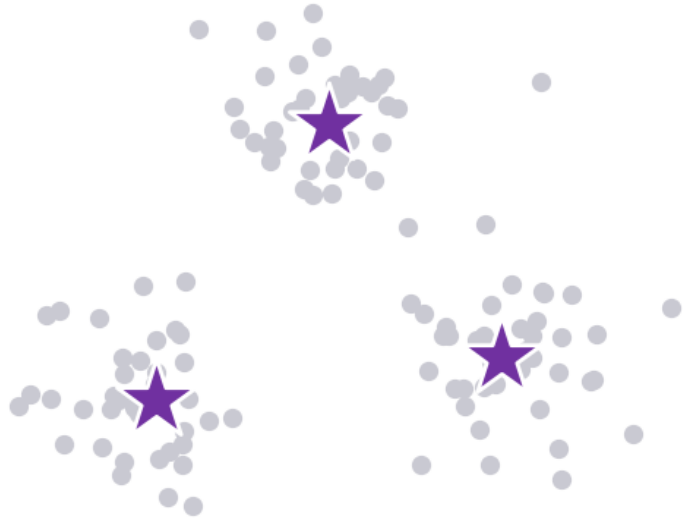
Variants

k-Modes (categorical), k-Prototypes (mixed), mini-batch, bisecting.

Exercise 2: *k*-Means by hand

	P1	P2	P3	P4	P5	P6	P7
x	1	1.5	3	5	3.5	4.5	3.5
y	1	2	4	7	5	5	4.5

1. Run *k*-Means with k = 2 and seeds **P1** and **P4** (Euclidean distance).
2. Give the clusters, the centroids and the **SSE** after every iteration.
3. How many iterations until it stops?



k-Means++

Careful seeding (Arthur and Vassilvitskii, 2007)

The idea: spread the seeds

1. Good seeds are **far apart**: one in each cluster.
2. Always taking the farthest point **picks the outliers**.
3. *k*-Means++: far points are chosen **with high probability**, not with certainty.

The *k*-Means++ seeding

1. Choose the first centre **uniformly at random**.
2. For every object x, compute **D(x)**: distance to the nearest chosen centre.
3. Choose the next centre with probability proportional to $\mathbf{D(x)^2}$.
4. Repeat 2 and 3 until k centres are chosen, then run *k*-Means.

$$P(x_i) = \frac{D(x_i)^2}{\sum_{j=1}^{n} D(x_j)^2}$$

The sum runs over all n objects; centres already chosen have D = 0 and cannot be picked again.

Example 1: the nine numbers again

x	2	3	4	10	11	12	20	25	30
$D(x)^2$	1	0	1	49	64	81	289	484	729
P(x)	0.001	0.000	0.001	0.029	0.038	0.048	0.170	0.285	0.429

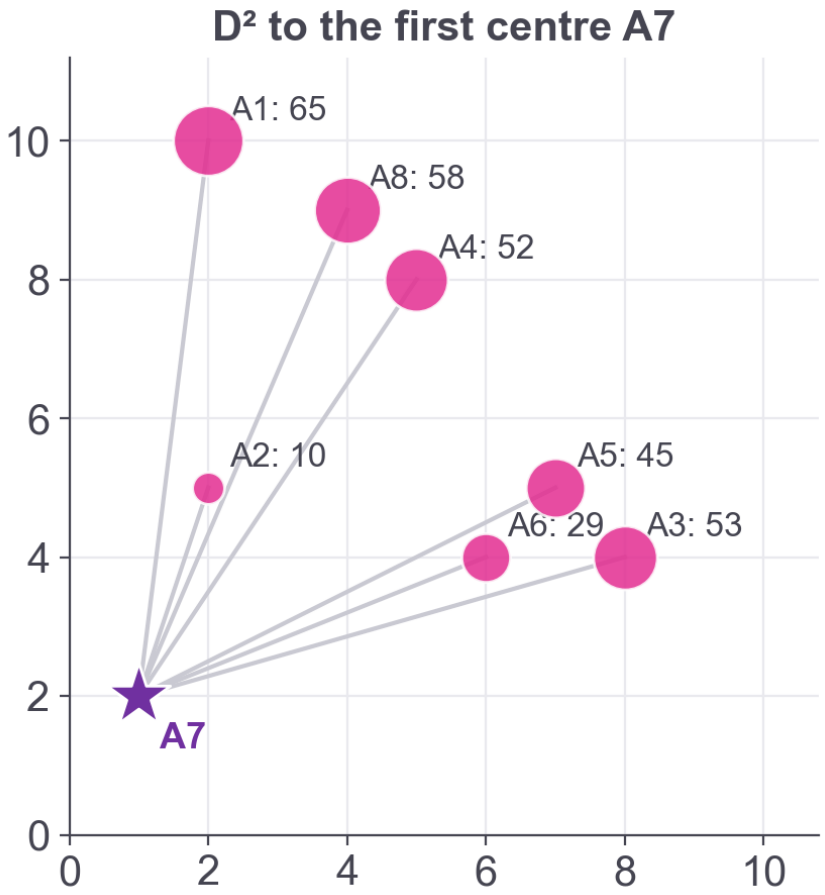
- First centre (random): **3**.
- 20, 25, 30 hold **88 %** of the probability.
- Random number r = 0.50 → second centre **25**.

Example 1: what good seeds buy

Seeds	Iterations	Final means	Final SSE
2 and 4 (arbitrary)	5	7 and 25	150
3 and 25 (*k*-Means++)	**2**	7 and 25	150

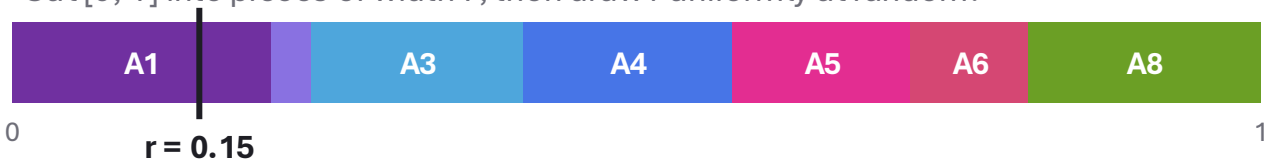
Same answer, **fewer iterations**: the first assignment is already the final clustering.

Example 2: the second centre



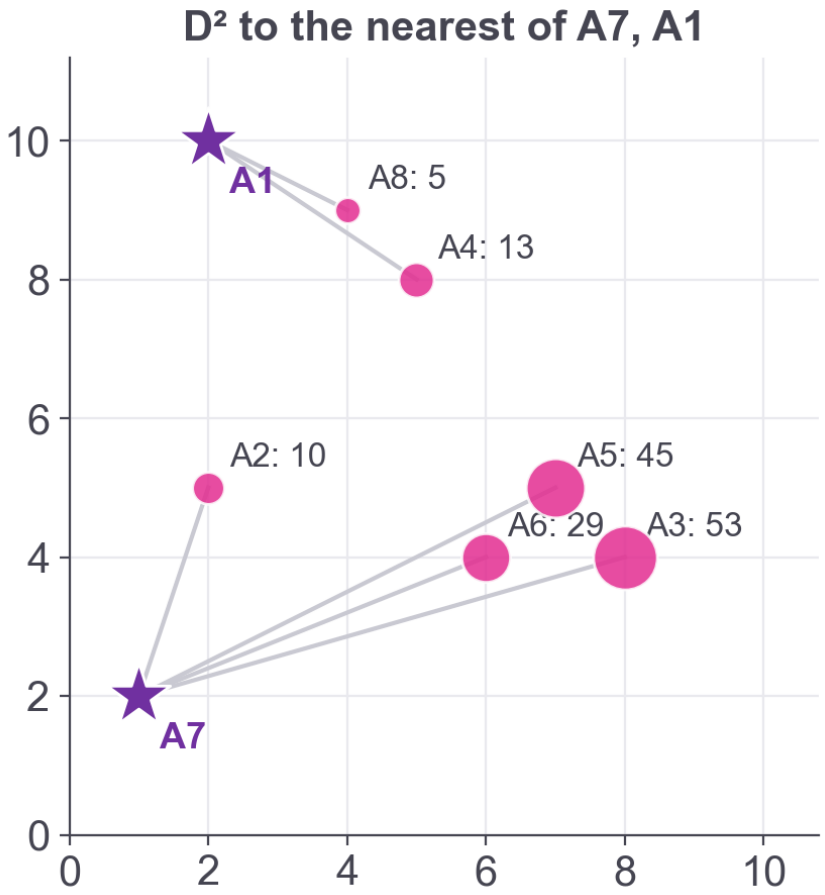
Point	D^2	P	cumulative
A1	65	0.208	0.208
A2	10	0.032	0.240
A3	53	0.170	0.410
A4	52	0.167	0.577
A5	45	0.144	0.721
A6	29	0.093	0.814
A7	0	0.000	0.814
A8	58	0.186	1.000

Cut [0, 1] into pieces of width P, then draw r uniformly at random:



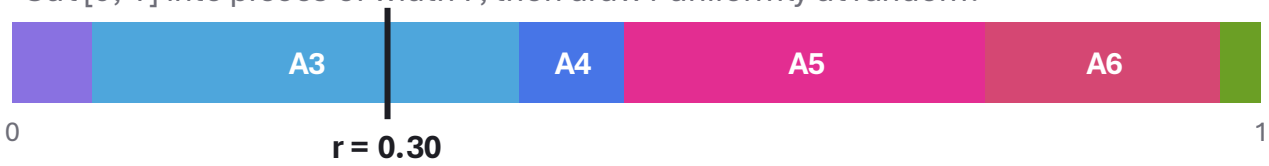
r lands in the piece of **A1**: the first row whose cumulative value is ≥ 0.15.

Example 2: the third centre



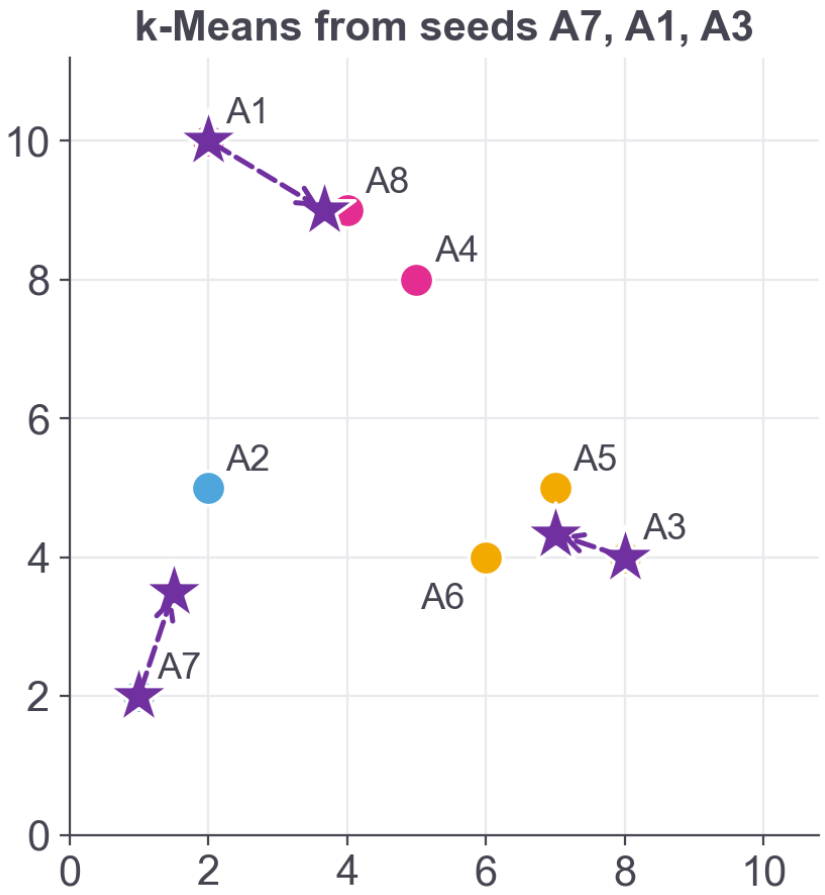
Point	D^2	P	cumulative
A1	0	0.000	0.000
A2	10	0.065	0.065
A3	53	0.342	0.406
A4	13	0.084	0.490
A5	45	0.290	0.781
A6	29	0.187	0.968
A7	0	0.000	0.968
A8	5	0.032	1.000

Cut [0, 1] into pieces of width P, then draw r uniformly at random:



r lands in the piece of **A3**: the first row whose cumulative value is ≥ 0.30.

Example 2: *k*-Means from the new seeds



Seeds	Iterations	Final SSE
A1, A4, A7	4	14.33
A7, A1, A3 (*k*-Means++)	**2**	14.33

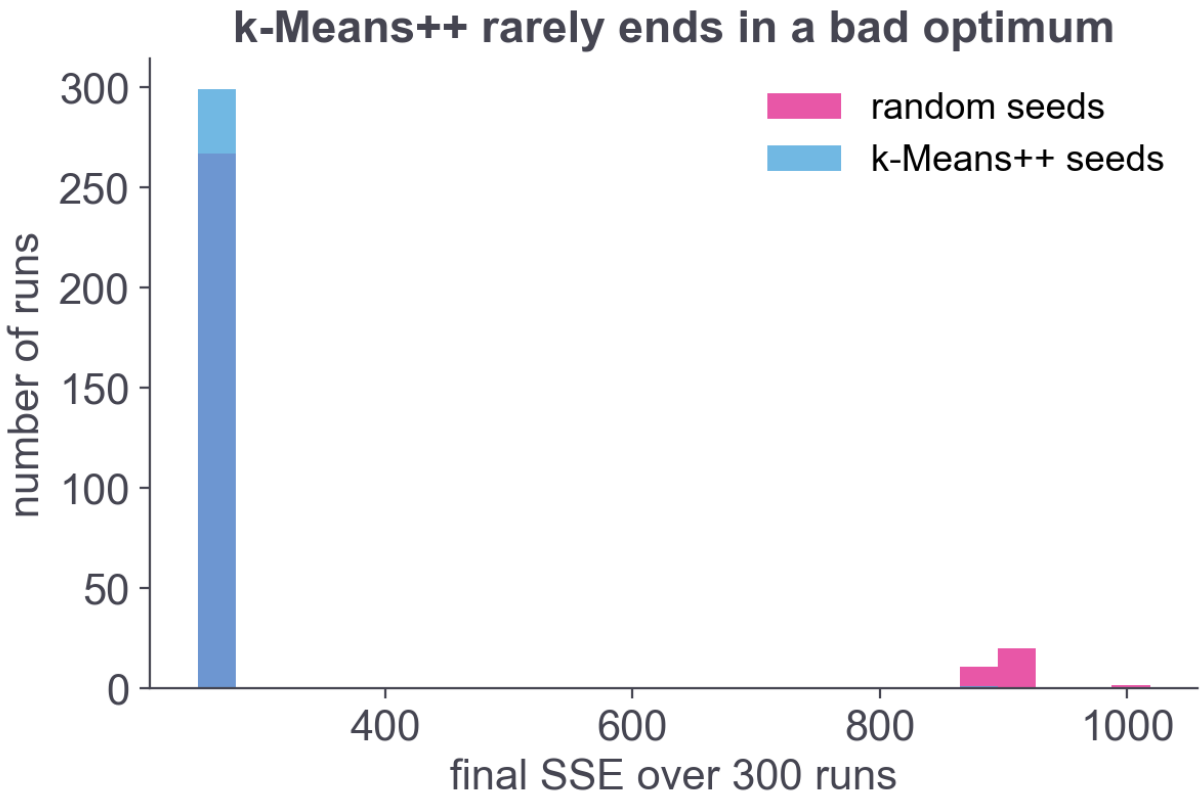
The first assignment is **already** the final clustering.

Is it only luck? The exact answer

Seeding	P(optimal SSE = 14.33)	Expected SSE	Expected iterations
uniform random	80.4 %	21.04	2.79
k-Means++	**91.8 %**	**17.13**	**2.35**

- All 336 possible seed choices for Example 2, weighted by their probability.
- *k*-Means++ is still random: **an unlucky draw remains possible**, only rarer.

A larger experiment



	random	*k*-Means++
optimum reached	89 %	**99.7 %**
mean SSE	332	**263**
mean iterations	5.7	**2.8**

Better clusterings **and** fewer iterations.

The guarantee

Arthur and Vassilvitskii (2007)

$$E[SSE] \leq 8\,(\ln k + 2) \cdot SSE_{opt}$$

- The first seeding with a **proven bound**.
- Random seeding can be arbitrarily bad.
- The default initialisation in scikit-learn, MATLAB, Spark.

Cost and practice

Cost

One pass per centre: $O(n \cdot k \cdot d)$, about one iteration.

Pay-off

Fewer iterations and a **lower SSE** on average.

Still random

Keep a few restarts.

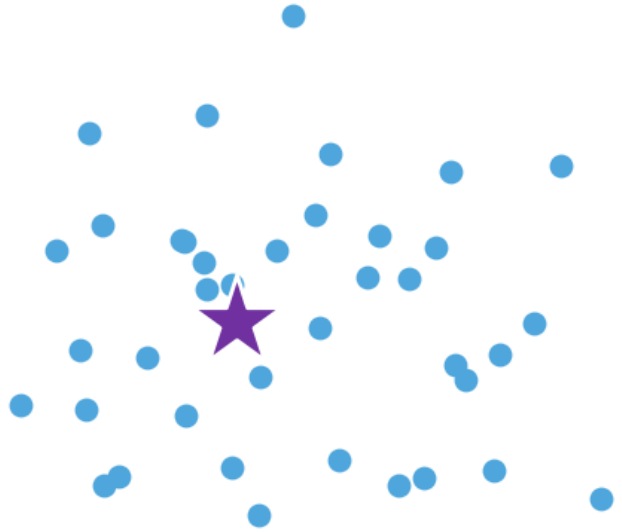
Very large data

k-Means‖: many centres per pass, in parallel.

Exercise 3: seed by hand

Data: 1, 2, 6, 7, 15 k = 3 first centre: 2

1. Compute $D(x)^2$ and the probability of every point.
2. With the random number r = 0.50, which point becomes the second centre?
3. Recompute the probabilities for the third centre.



k-Medoids

Real objects as cluster centres

The trouble with means



1. The mean is **not robust**: one outlier drags it from 2.5 to 22.
2. The mean is usually **not a real object**.
3. The mean needs **numeric** attributes.

Medoids

Medoid: the most central object

$$m_i = \operatorname{argmin}_{o \in C_i} \sum_{p \in C_i} d(p, o)$$

Total cost (absolute error)

$$E = \sum_{i=1}^{k} \sum_{p \in C_i} d(p, m_i)$$

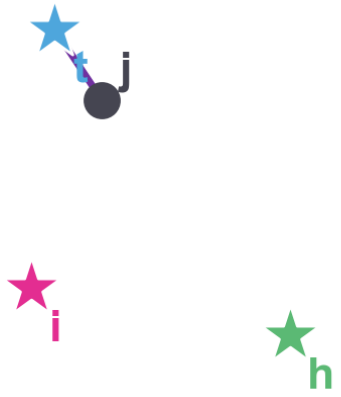
- The centre is an **actual object** of the data set.
- Works with **any dissimilarity**: Manhattan, Jaccard, edit distance…
- Only the dissimilarity matrix is needed.

PAM: Partitioning Around Medoids

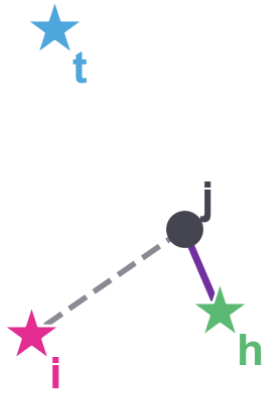
1. Choose **k objects** as the initial medoids.
2. Assign every object to its **nearest medoid**.
3. For each medoid m and non-medoid h:
   compute **S = E(h replaces m) – E**.
4. If the best swap has **S < 0**, perform it and go to 2. Otherwise stop.

The cost of a swap

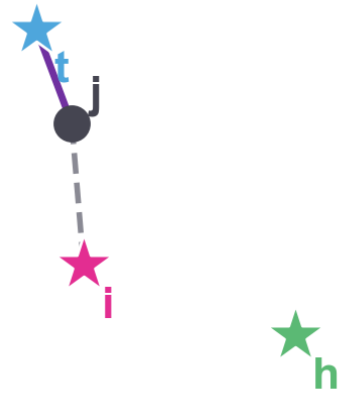
1. j stays with another medoid t



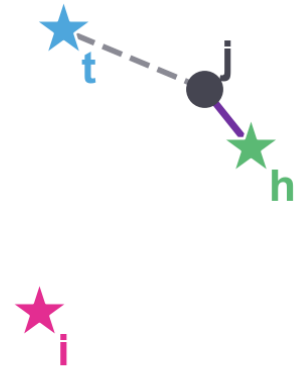
2. j moves from i to h



3. j moves from i to t



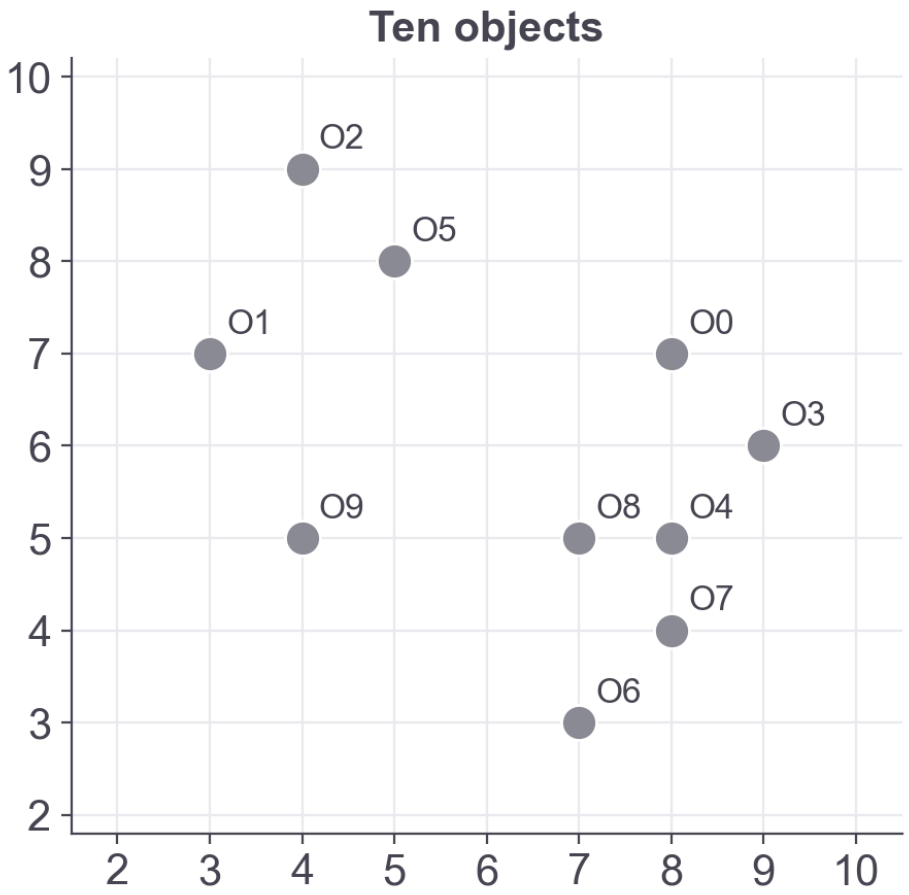
4. j moves from t to h



$$TC_{ih} = \sum_j C_{jih}$$

Replacing medoid *i* by *h* changes the cost of every object *j* in one of **four ways**.

Example 1: ten objects, two medoids

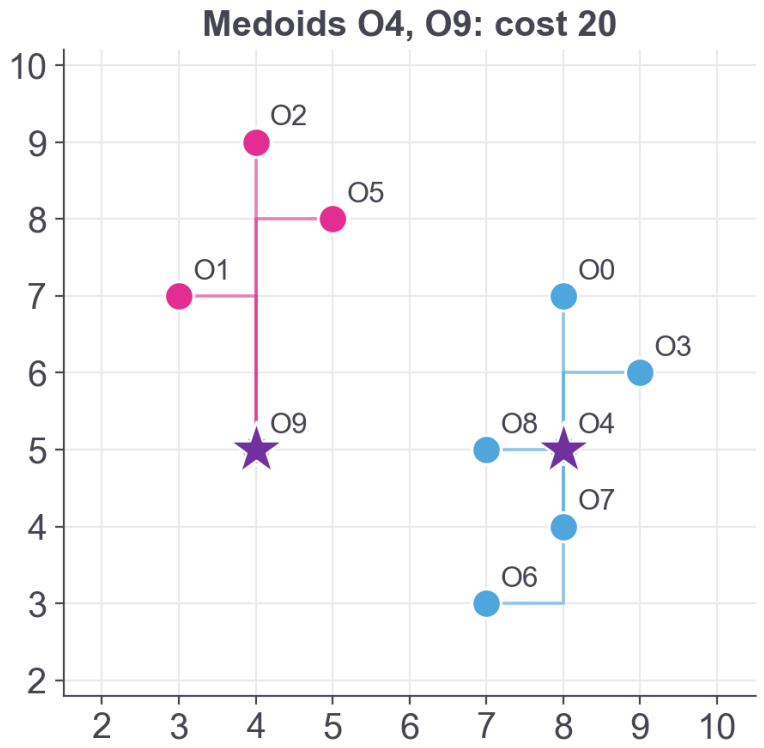


	x	y		x	y
O0	8	7	O5	5	8
O1	3	7	O6	7	3
O2	4	9	O7	8	4
O3	9	6	O8	7	5
O4	8	5	O9	4	5

- **Manhattan** distance, k = 2.
- Initial medoids: **O4** and **O9**.

Example 1: assign to the nearest medoid

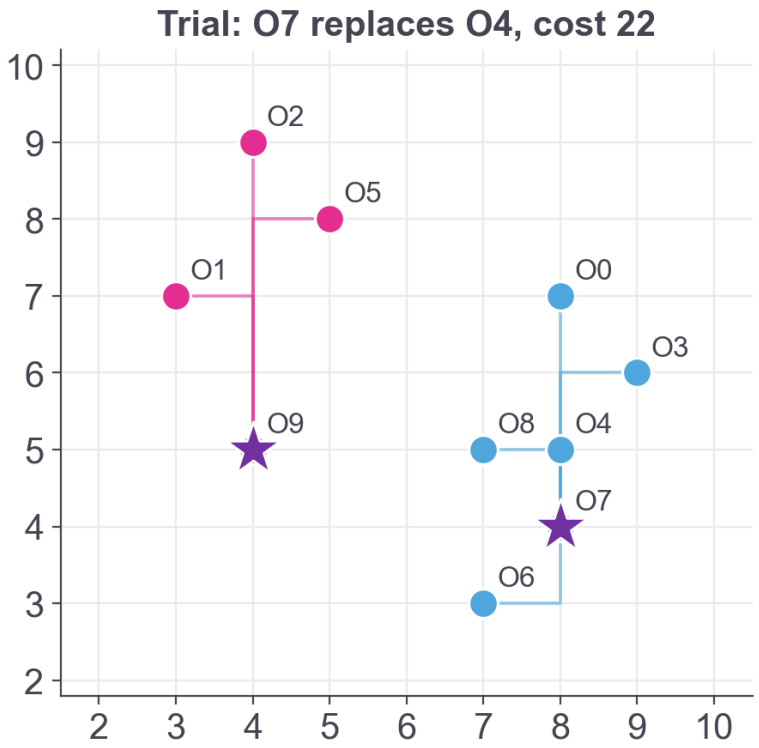
	d(O4)	d(O9)	cluster
O0	2	6	C1
O1	7	3	C2
O2	8	4	C2
O3	2	6	C1
O4	medoid	4	C1
O5	6	4	C2
O6	3	5	C1
O7	1	5	C1
O8	1	3	C1
O9	4	medoid	C2



Cost = (2+2+3+1+1) + (3+4+4) = **20**

Example 1: try a swap

	d(O7)	d(O9)	cluster
O0	3	6	C1
O1	8	3	C2
O2	9	4	C2
O3	3	6	C1
O4	1	4	C1
O5	7	4	C2
O6	2	5	C1
O7	medoid	5	C1
O8	2	3	C1
O9	5	medoid	C2



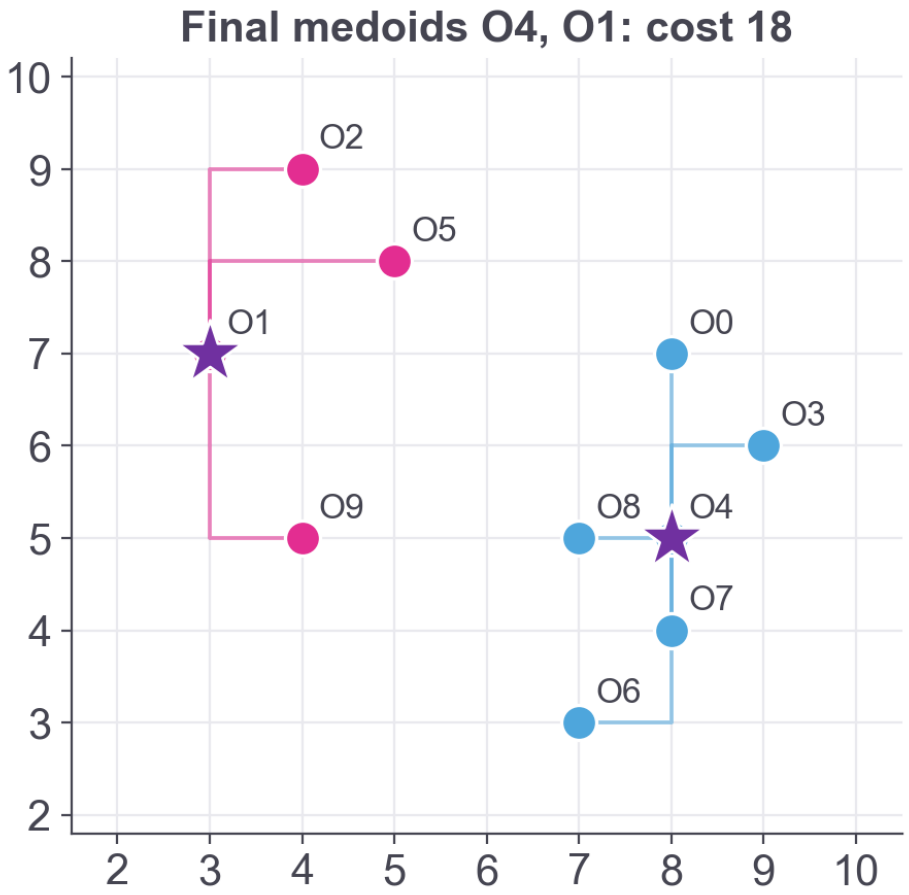
S = 22 − 20 = +2: **reject**, keep O4

Example 1: all sixteen swaps

by →	O0	O1	O2	O3	O5	O6	O7	O8
replace O4	26	34	34	26	32	28	22	22
replace O9	26	18	18	32	18	31	32	28

- Current cost: 20.
- Best swap: O9 → **O1**, cost **18**, S = −2.
- Ties with O2 and O5: take the first.

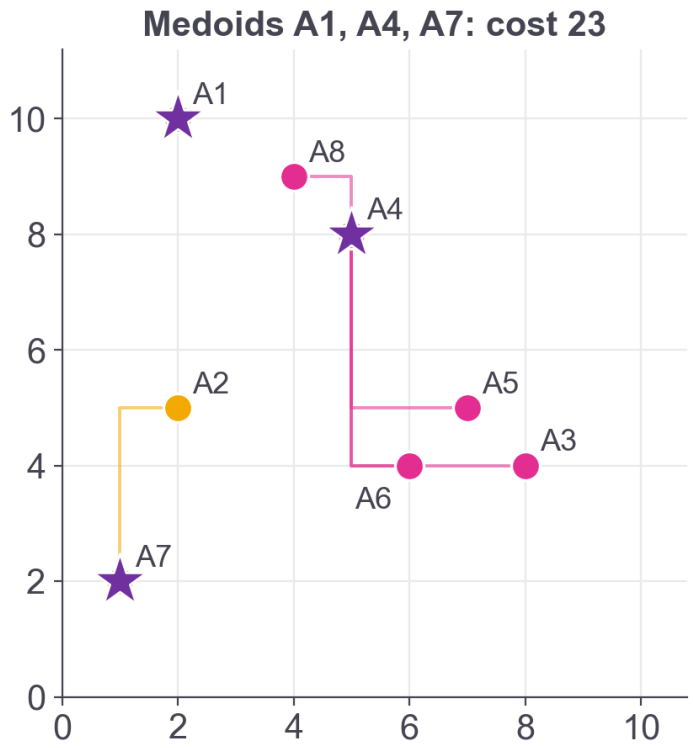
Example 1: the result



- Second round: **no swap** lowers the cost → stop.
- Medoids: **O4** and **O1**, cost **18**.
- C1 = {O0, O3, O4, O6, O7, O8}
- C2 = {O1, O2, O5, O9}

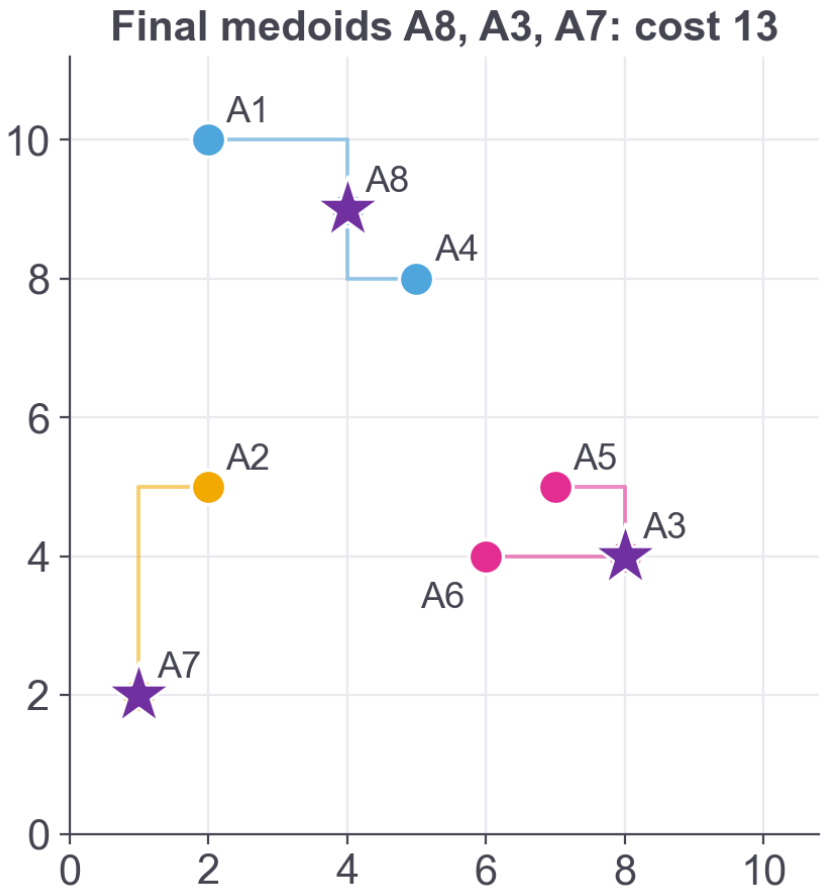
Example 2: the eight points with PAM

	d(A1)	d(A4)	d(A7)	cluster
A1	0	5	9	C1
A2	5	6	4	C3
A3	12	7	9	C2
A4	5	0	10	C2
A5	10	5	9	C2
A6	10	5	7	C2
A7	9	10	0	C3
A8	3	2	10	C2



Manhattan, k = 3.
Cost = **23**

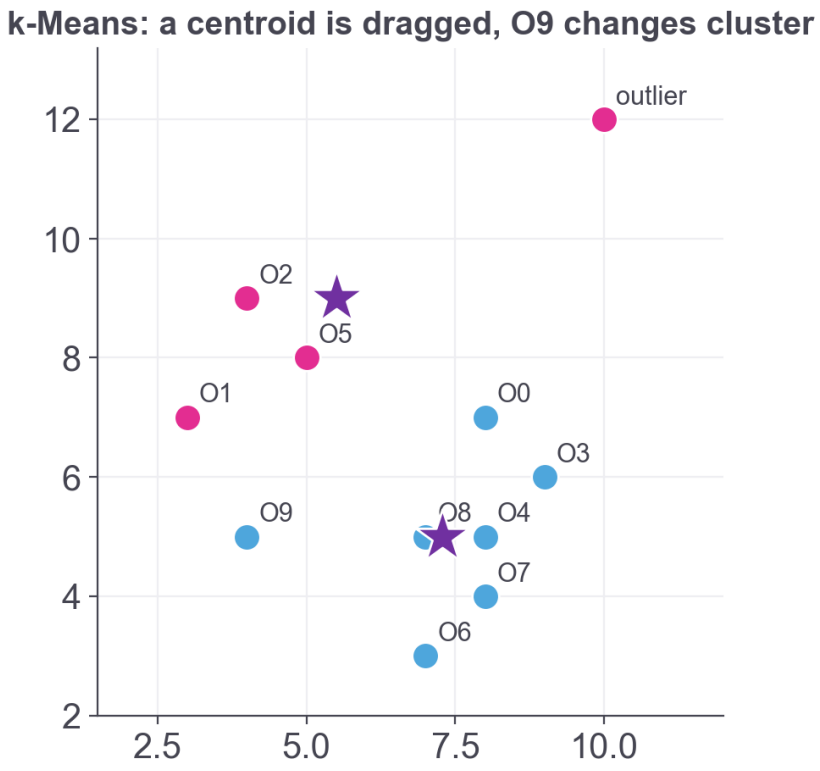
Example 2: two swaps to the end

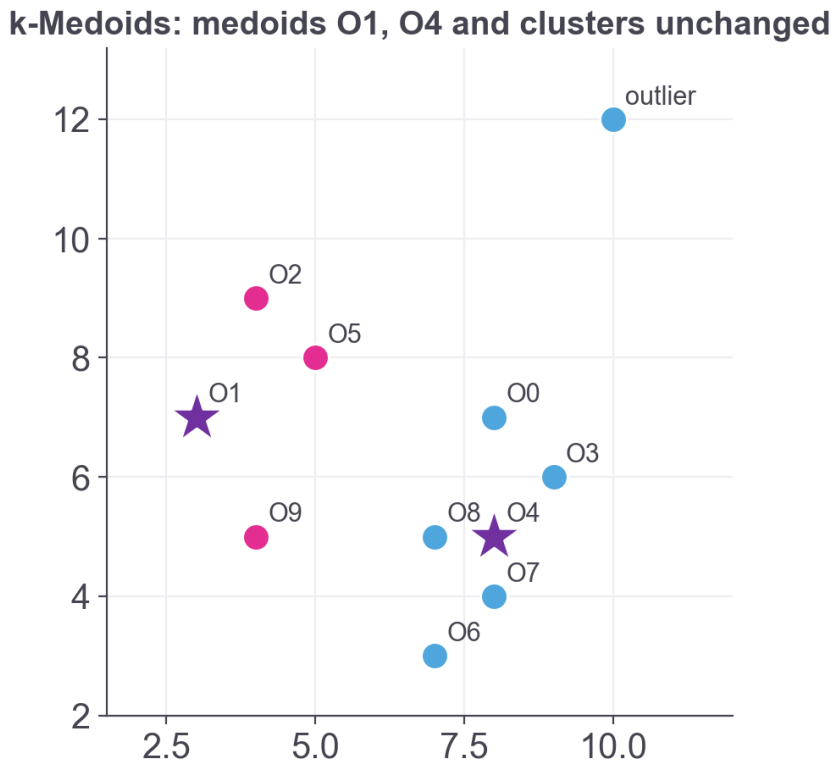


Round	Medoids	Cost	Best swap	S
1	A1, A4, A7	23	A1 → A3	−8
2	A3, A4, A7	15	A4 → A8	−2
3	A3, A8, A7	**13**	none	stop

- The **same clusters** as *k*-Means.
- The centres are now **real points**: A8, A3, A7.

Example 3: add an outlier





One outlier at (10, 12): *k*-Means **reshuffles** the clusters, *k*-Medoids **does not**.

Cost of PAM

PAM

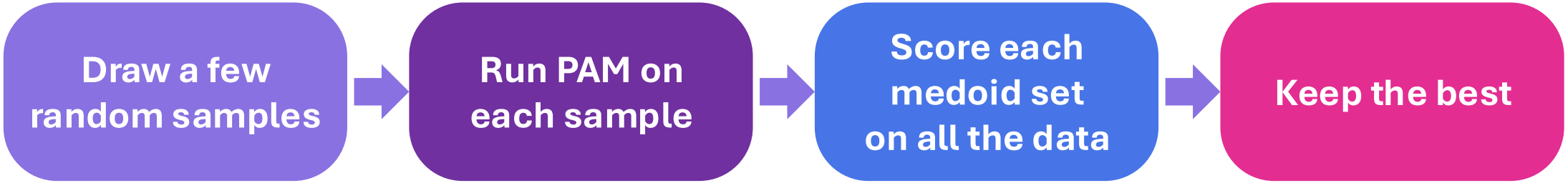
$$O(k\,(n-k)^2) \text{ per iteration}$$

n = 10 000, k = 10	per iteration
k-Means	$\approx 10^5$ distances
PAM	$\approx 10^9$ distances

PAM does not scale.

Use **sampling**: CLARA, CLARANS.

CLARA: sample, then PAM



Strength

Handles much larger data sets than PAM.

Weakness

A biased sample hides the best medoids: quality depends on the sample.

CLARANS: randomised search

1. A solution is a **set of k medoids**; its neighbours differ in one medoid.
2. Check a **random sample** of neighbours; move to a better one when found.
3. At a local optimum, **restart** from a random solution; keep the best.

k-Means versus *k*-Medoids

	k-Means	*k*-Medoids (PAM)
Centre	mean (may not exist in the data)	an actual object
Objective	sum of squared Euclidean distances	sum of dissimilarities
Data	numeric	anything with a dissimilarity
Outliers	sensitive	robust
Cost per iteration	$O(n \cdot k \cdot d)$	$O(k(n-k)^2)$

Exercise 4: PAM by hand

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
x	2	3	3	4	6	6	7	7	8	7
y	6	4	8	7	2	4	3	4	5	6

1. Manhattan distance, k = 2, medoids **X2** and **X8**: assign the objects and compute the cost.
2. Is replacing X8 by **X7** an improvement? Compute S.
3. Find one swap that lowers the cost, if any.



Wrapping up

Evaluate, compare, look beyond

Evaluating a clustering

Internal: no labels

- **SSE**: cohesion inside clusters
- Separation between clusters
- **Silhouette**: both at once

External: labels exist

- **Purity**: share of the majority class
- Entropy of classes per cluster
- Rand index

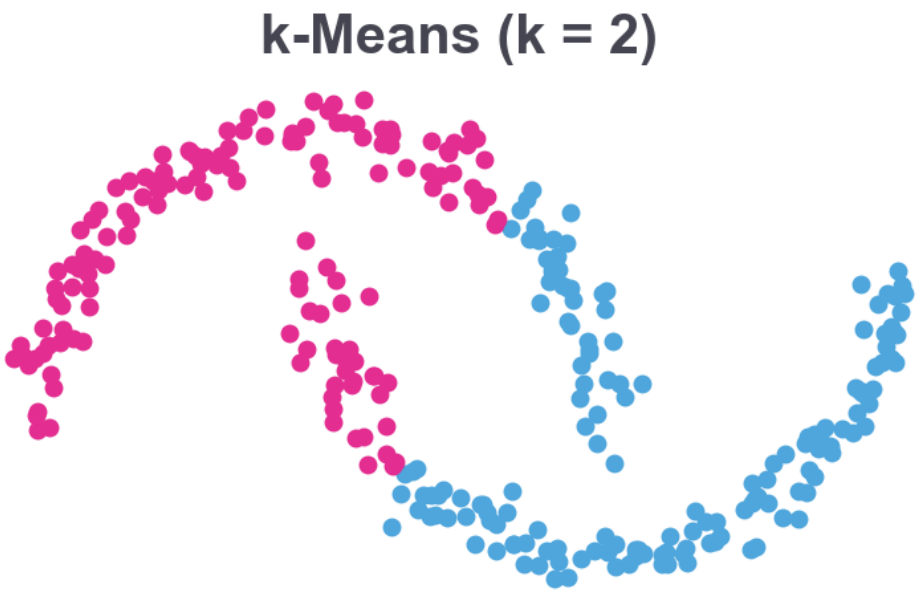
Every algorithm finds clusters, even in random data.

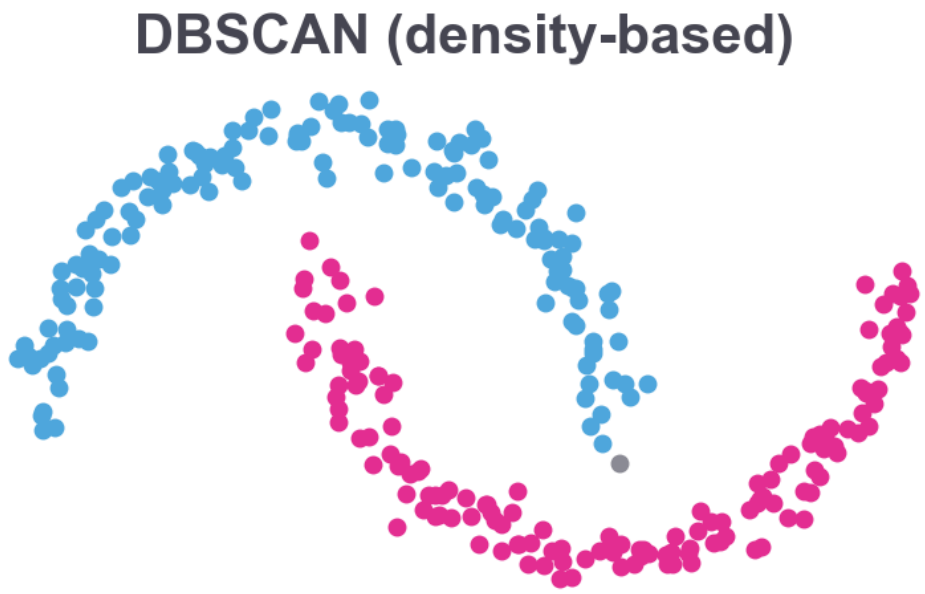
Always validate.

The three algorithms side by side

	k-Means	*k*-Means++	*k*-Medoids (PAM)
Centre	mean	mean	actual object
Seeding	random	D^2 sampling	random
Outliers	sensitive	sensitive	robust
Cost	$O(t \cdot n \cdot k \cdot d)$	+ $O(n \cdot k \cdot d)$ seeding	$O(k(n - k)^2)$ per iteration
Use when	numeric, large data	the default choice	outliers, any dissimilarity, small n

Beyond partitioning





Non-globular clusters need another idea: **density** (DBSCAN) or a **hierarchy**.

Summary

- Clustering finds groups **without labels**; the result depends on the **distance**.
- **k-Means** alternates assign and update to lower the SSE; it stops at a local optimum.
- **k-Means++** seeds by D^2 sampling: better results, fewer iterations.
- **k-Medoids** uses real objects as centres: robust, general, but costly.
- Choose **k** with the elbow or the silhouette, and always validate.

Homework

1. Run *k*-Means on the ten objects O0…O9 (k = 2, seeds O4 and O9). Compare with the PAM result.

2. Run PAM on the eight points A1…A8 with k = 2 (Manhattan, medoids A1 and A7).

3. Compute the silhouette coefficient of A1 in the final clustering of Example 2.

4. With scikit-learn, compare random and *k*-Means++ initialisation on the Iris data over 100 runs.

References

Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.), ch. 10. Morgan Kaufmann.
Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). Introduction to Data Mining (2nd ed.), ch. 7. Pearson.
Zaki, M. J., & Meira, W. (2020). Data Mining and Machine Learning (2nd ed.), ch. 13. Cambridge University Press.
Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of Massive Datasets (3rd ed.), ch. 7. Cambridge University Press.
MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. 5th Berkeley Symposium, 281–297.
Lloyd, S. P. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137.
Arthur, D., & Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. SODA '07, 1027–1035.
Kaufman, L., & Rousseeuw, P. J. (1990). Finding Groups in Data: An Introduction to Cluster Analysis. Wiley.
Ng, R. T., & Han, J. (1994). Efficient and effective clustering methods for spatial data mining. VLDB '94, 144–155.
Huang, Z. (1998). Extensions to the k-means algorithm for clustering large data sets with categorical values. DMKD, 2(3), 283–304.


Q&A